第11章 大数据与云数据管理

西安交通大学计算机科学与技术学院　　任雪斌
xuebinren@mail.xjtu.edu.cn

目录

11.1 大数据

11.2 云计算

11.3 云数据管理

11.4 No SQL数据库

11.1 大数据

➢目前IT领域最热的5个热词是什么?

- 人工智能(AI)(机器学习、深度学习)
- 大数据
- 云计算
- 物联网
- 区块链
- 其他：移动互联网、社交网、XXX2.0/3.0/...

11.1 大数据

➢大数据(Big Data)一词的起源

- 2008年，在Google成立10周年之际，《Nature》出版一期专刊，专门讨论未来的大数据(Big Data)处理相关的一系列技术问题和挑战。
- 从此，Big Data成为一个热词开始广泛传播！
- 所谓“大数据”，泛指规模达到几百TB，甚至PB级的数据，广泛出现在科学研究、Web集成、多媒体等诸多领域中。
- “大数据”是伴随数据获取技术的发展，Web 2.0、WSN(传感网)和CPS(物联网)等应用的快速普及而提出的概念。

11.1 大数据

➢与大数据相关联的其它几个词汇

- “超大规模数据库”(VLDB)这个词是20世纪70年代中期出现(VLDB会议于1975年发起)。当时，数百万条记录即是超大规模。
- “海量Massive数据”则是21世纪初出现，指越来越多的应用建立在多源数据集成基础之上时，数据规模急剧扩大，数据类型由关系数据发展到非结构化、半结构化数据。

11.1 大数据

➢与大数据相关联的其它几个词汇

- “非结构化数据”是指在获得数据之前无法预知其结构的数据，如文本、图像、视频等。其包含数千万个文档、数百万张照片或者工程设计图的数据集成为常态，现有的关系数据库已无力有效管理这些数据，如何快速访问数据成为核心挑战。
- 为此，以Hadoop为代表的分布式文件系统和MapReduce计算框架应运而生。

11.1 大数据

➢什么是大数据?

- 没有统一定义，因为“大”是一个与能力和需求相关的概念，没有一个绝对统一的量化标准。
- 与大数据(Big Data)相类似的词汇：
  - Large Scale 大规模
  - Very Large 超大规模
  - Massive 海量
  - …
- 显然，争论它们中哪一个更大没有什么意义。

11.1 大数据

➢什么是大数据?

- 从历史的视角去看这几个词:
  - “Very Large”表示GB级别的数据。
  - “Massive”表示TB级的数据。
  - “Big Data”则是PB级别及其以上的数据。

字节的次方单位

十进制前缀 (SI)			二进制前缀 (IEC 60027-2)		
名字	缩写	次方	名字	缩写	次方
千字节	KB	10^3	kibibyte	KiB	210
兆字节	MB	10^6	mebibyte	MiB	220
吉字节	GB	10^9	gibibyte	GiB	230
太字节	TB	10^{12}	tebibyte	TiB	240
拍字节	PB	10^{15}	pebibyte	PiB	250
艾字节	EB	10^{18}	exbibyte	EiB	260
泽字节	ZB	10^{21}	zebibyte	ZiB	270
尧字节	YB	10^{24}	yobibyte	YiB	280

11.1 大数据

➢什么是大数据?

- 说法1：大数据是指数量特别巨大的信息集合，数据量达到PB、EB或ZB级别。

- 问题：将“大数据”简单等同于“大量数据”，但数据量巨大只是大数据的表面特征，而非其本质特征。

11.1 大数据

➢什么是大数据?

- 说法2：大数据是指具有类型繁多、不同结构化程度、不均衡价值密度、不一致动态特征、不同应用处理特征等特点的巨大信息集合。

- 问题：没有说明大数据为什么数量巨大、类型多样。

11.1 大数据

➢什么是大数据?

- 说法3：大数据是指无法在可容忍的时间内用传统方法或工具进行获取、存储、检索、共享、分析和可视化的数据集合。数据集组成非常巨大和复杂，以至于传统数据库管理工具处理起来面临很多问题。

* from {Big Data} @wikipedia.

- 问题：虽然认识到传统方法在大数据面前已无能为力，但仍然没有抓住大数据的本质特征。

11.1 大数据

➢难道这就是大数据?



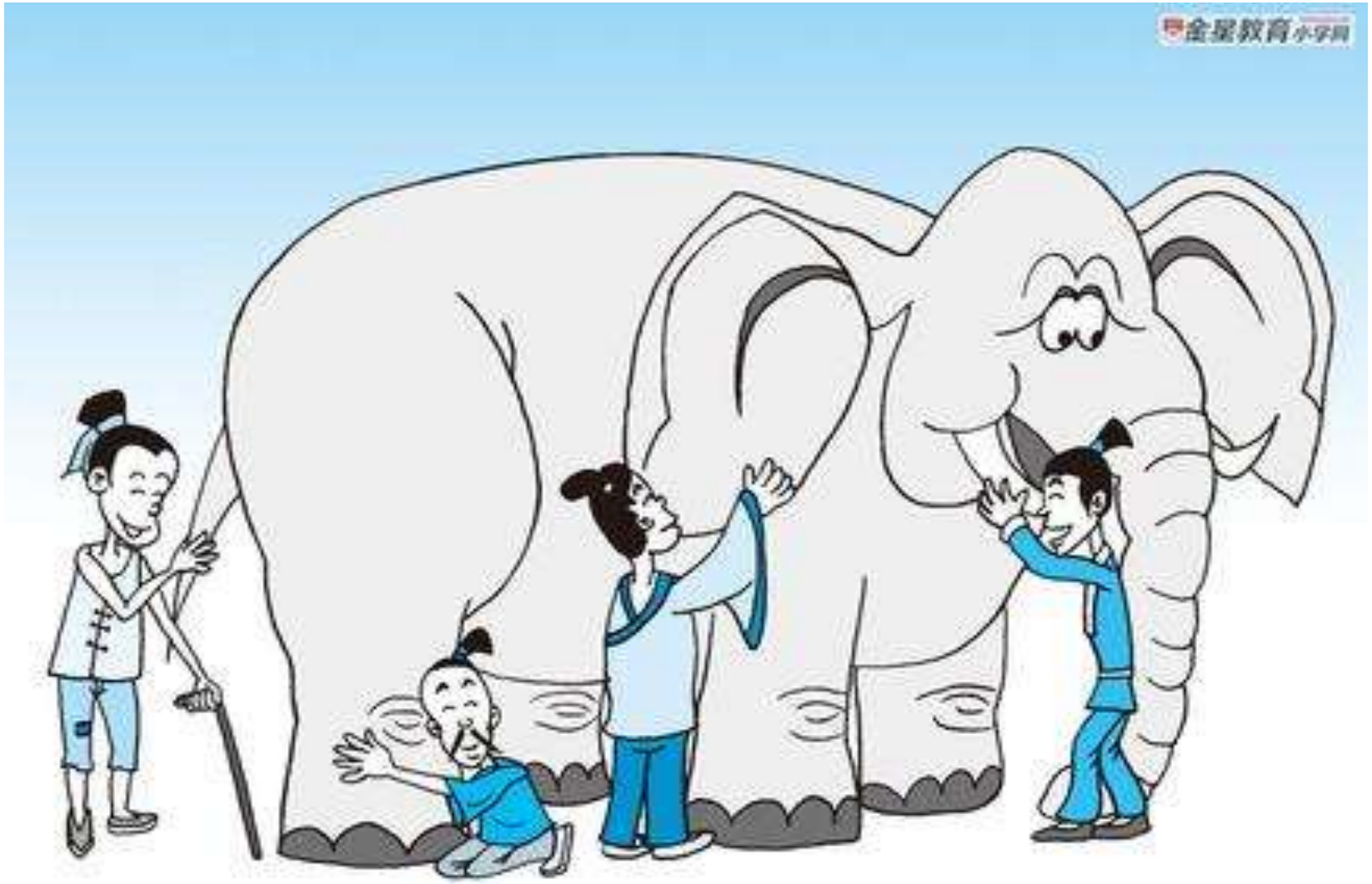
11.1 大数据

➢难道这就是大数据?



11.1 大数据

➢难道这就是大数据?



11.1 大数据

➢大数据有何本质特征?

- 要解决大数据面临的问题，就必须抓住大数据的本质特征和大数据处理面临的主要矛盾，然后针对这些本质特征寻找有效解决之道。

11.1 大数据

➢大数据有何本质特征?

- 特征1：面向开放世界，没有清晰边界；
- 特征2：问题没有确定解，弱化一致性要求；
- 特征3：大数定律失效，需要覆盖全样本空间；
- 特征4：多维度、结构复杂，前期处理代价极大；
- 特征5：价值密度稀疏，必须通过聚合产生价值；
- 特征6：数量巨大，人力已根本无法处理，必须通过机器自动处理；
- 特征7：...

11.1 大数据

➢什么是大数据

- 大数据是由于面向开放世界(如互联网)而导致的数量极其庞大，结构杂乱无章，分布异构、语义模糊的信息集合。

11.1 大数据

➢第三次信息化浪潮

- 根据IBM前首席执行官郭士纳的观点，IT领域每隔十五年就会迎来一次重大变革

表1-1 三次信息化浪潮

信息化浪潮	发生时间	标志	解决问题	代表企业
第一次浪潮	1980年前后	个人计算机	信息处理	Intel、AMD、IBM、苹果、微软、联想、戴尔、惠普等
第二次浪潮	1995年前后	互联网	信息传输	雅虎、谷歌、阿里巴巴、百度、腾讯等
第三次浪潮	2010年前后	物联网、云计算和大数据	信息爆炸	将涌现出一批新的市场标杆企业

11.1 大数据

➢信息科技为大数据时代提供技术支撑

- 存储设备容量不断增加、价格不断下降

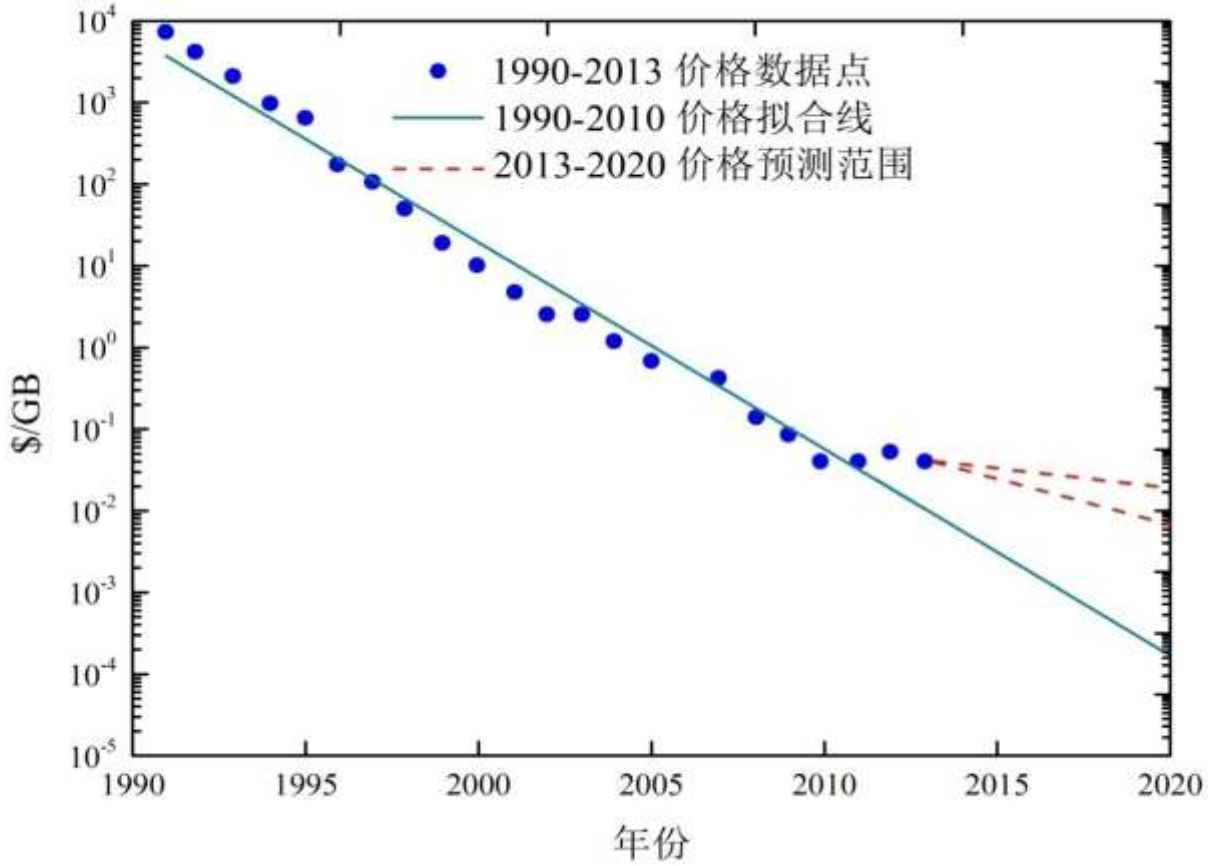


图1-1 存储价格随时间变化情况

11.1 大数据

➢信息科技为大数据时代提供技术支撑

- CPU处理能力大幅提升

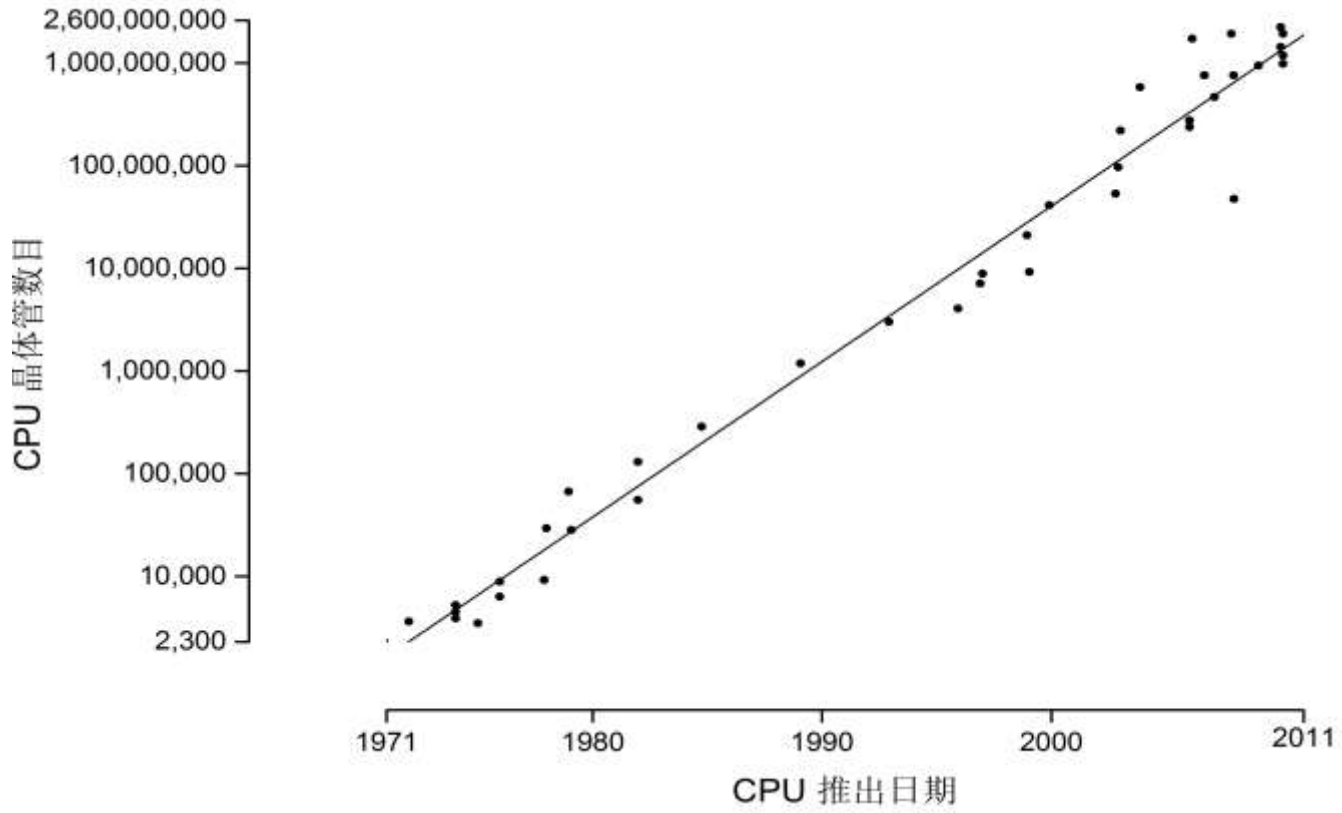


图1-3 CPU晶体管数目随时间变化情况

11.1 大数据

➢信息科技为大数据时代提供技术支撑

- 网络带宽不断增加

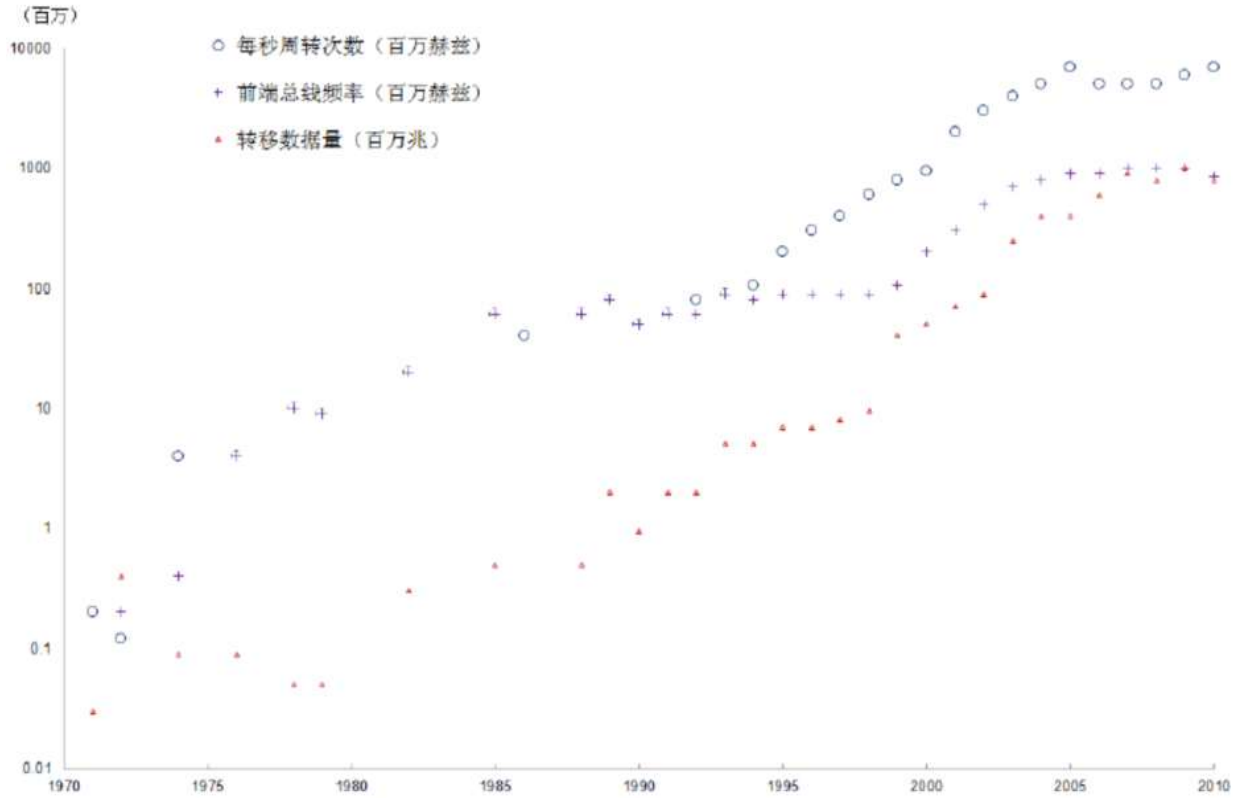


图1-4 网络带宽随时间变化情况

11.1 大数据

➢数据产生方式的变革促成大数据时代的来临



被动产生—>主动产生—>自动产生

图1-5 数据产生方式的变革

11.1 大数据

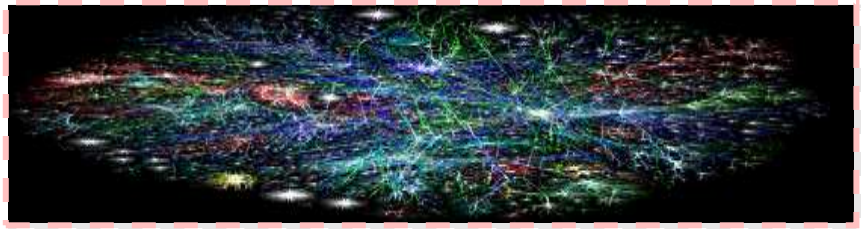
➢大数据的发展历程

表1-2 大数据发展的三个阶段

阶段	时间	内容
第一阶段：萌芽期	上世纪90年代至本世纪初	随着数据挖掘理论和数据库技术的逐步成熟，一批商业智能工具和知识管理技术开始被应用，如数据仓库、专家系统、知识管理系统等。
第二阶段：成熟期	本世纪前十年	Web2.0应用迅猛发展，非结构化数据大量产生，传统处理方法难以应对，带动了大数据技术的快速突破，大数据解决方案逐渐走向成熟，形成了并行计算与分布式系统两大核心技术，谷歌的GFD和MapReduce等发数据技术受到追捧，Hadoop平台开始大行其道。
第三阶段：大规模应用期	2010年以后	大数据应用渗透各行各业，数据驱动决策，信息社会智能化程度大幅提高。

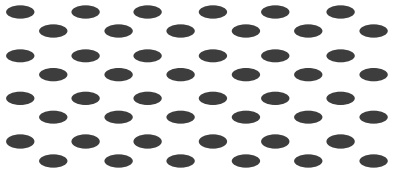
11.1 大数据

➢大数据的4V特性





Volume

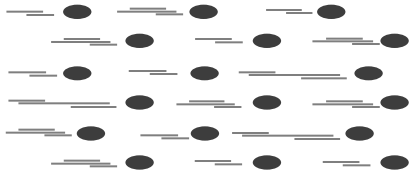


体量大

IDC Analyze the Future

到2020年，数据总量达40ZB，人均5. 2TB

Velocity

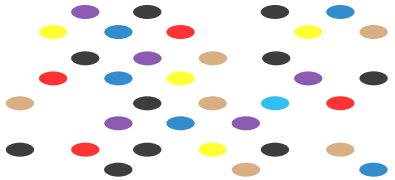


速度快

facebook

分享的内容条目超过25亿个/天，增加数据超过500TB/天

Variety



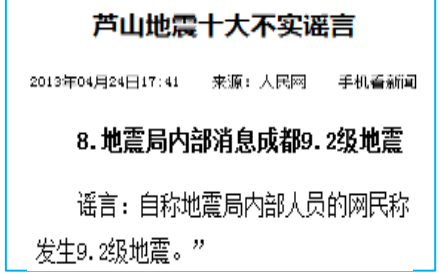
模态多



Veracity



真伪难辨



芦山地震十大不实谣言

2013年04月24日17:41　来源：人民网　手机看新闻

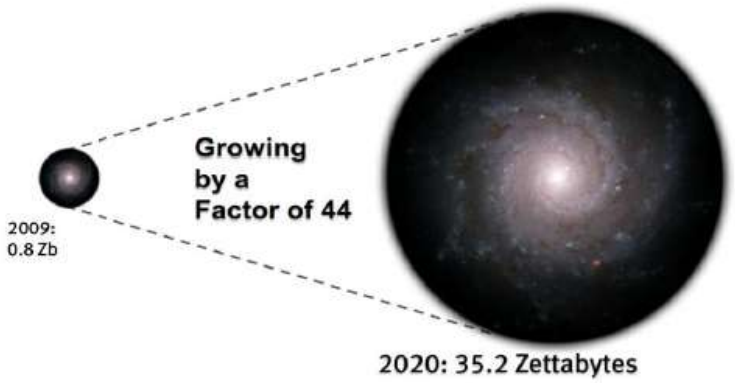
8. 地震局内部消息成都9. 2级地震

谣言：自称地震局内部人员的网民称发生9. 2级地震。”

11.1 大数据

➢数据量大

- 根据IDC作出的估测，人类掌握的数据一直都在以每年50%的速度增长，也就是说每两年就增长一倍(大数据摩尔定律)。
- 人类在最近两年产生的数据量相当于之前产生的全部数据量。
- 预计到2020年，全球将总共拥有35ZB的数据量，相较于2010年，数据量将增长近30倍



TERABYTE	10 的 12 次方	一块 1TB 硬盘		200,000 照片或 mp3 歌曲
PETABYTE	10 的 15 次方	两个数据中心机柜		16 个 Blackblaze pod 存储单元
EXABYTE	10 的 18 次方	2,000 个机柜		占据一个街区的 4 层数据中心
ZETTABYTE	10 的 21 次方	1000 个数据中心		纽约曼哈顿的 1/5 区域
YOTTABYTE	10 的 24 次方	一百万个数据中心		特拉华州和罗德岛州

11.1 大数据

➢数据类型繁多

- 大数据是由结构化和非结构化数据组成的
  - 10%的结构化数据，存储在数据库中。
  - 90%的非结构化数据，它们与人类信息密切相关。
- 非结构化数据类型多样
  - 邮件、视频、微博。
  - 位置信息、链接信息。
  - 手机呼叫、网页点击。
  - “长微博”。
  - …



11.1 大数据

➢快速处理

- 从数据的生成到消耗，时间窗口非常小，可用于生成决策的时间非常少。
- 1秒定律：这一点也是和传统的数据挖掘技术有着本质的不同。



11.1 大数据

➢价值密度低

- 价值密度低，商业价值高
- 以视频为例，连续不间断监控过程中，可能有用的数据仅仅有一两秒，但是具有很高的商业价值。





11.1 大数据

➢大数据与常规数据



特性

- 范围广
- 模态多
- 增长快
- 关联繁
- 时序长

问题

- 稠密与稀疏共存
- 冗余与缺失并存
- 动态与静态互现
- 显式与隐藏均有

挑战

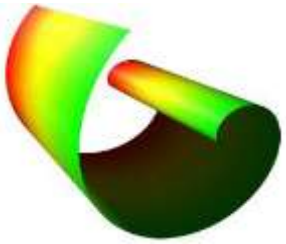
- 描述与存储的挑战
- 分析与理解的挑战
- 挖掘与预测的挑战



- 规模较小
- 模态有限
- 增速较慢
- 关联相对简单
- 持续时间较短

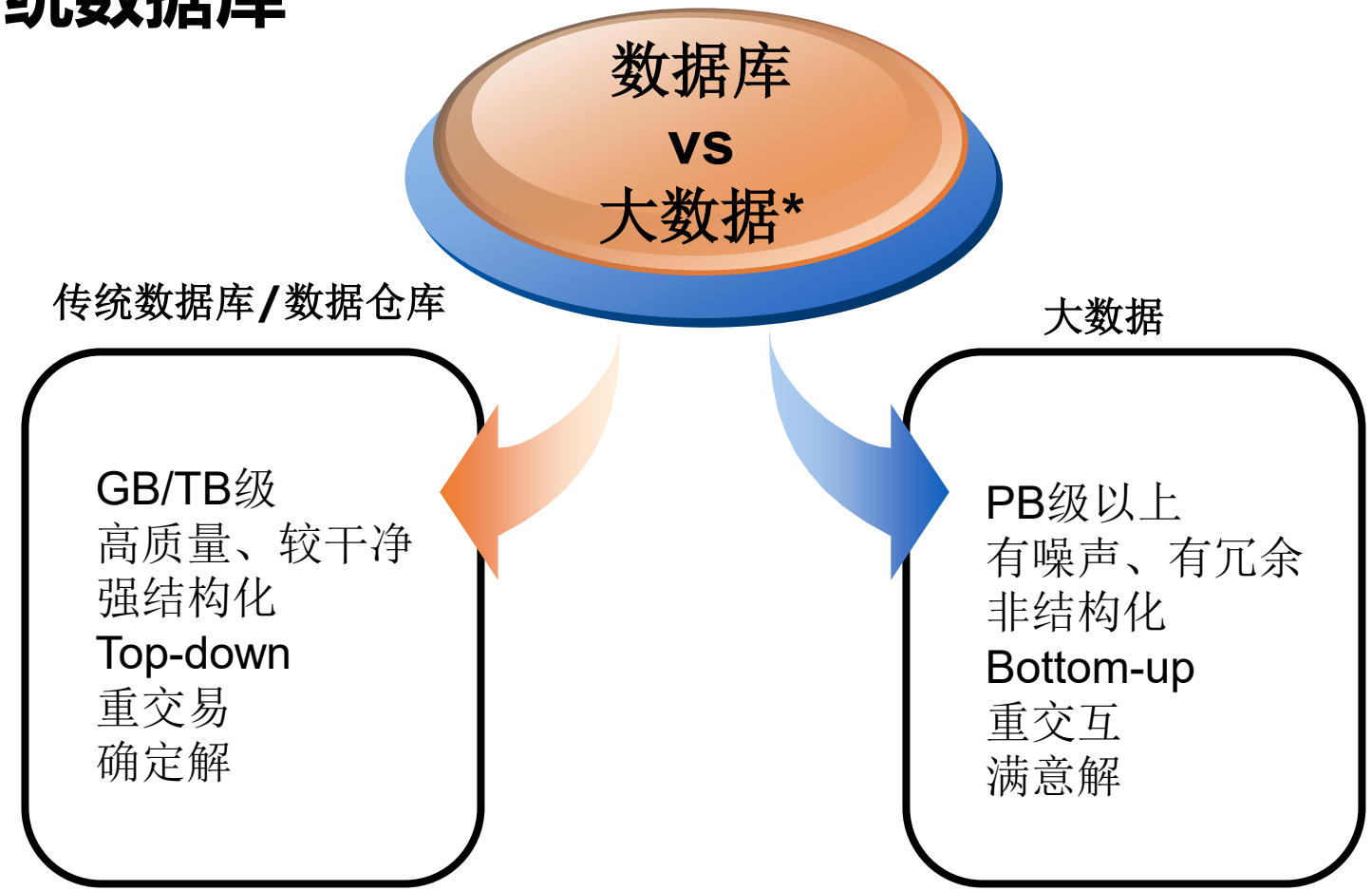
- 数据结构相对简单
- 处理方法通常为模型化、参数化
- 应用目标相对明确





11.1 大数据

➢大数据与传统数据库



* 摘自李德毅院士于2012年5月23日在云计算大会上的讲话，略有修改

11.1 大数据

➢大数据涉及诸多不同的领域


天文


气象


基因

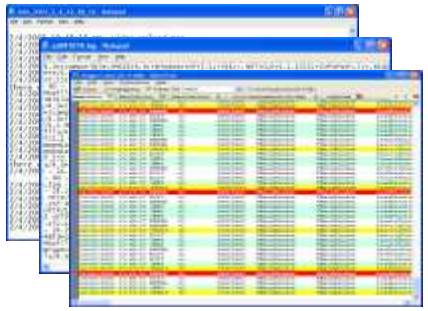

医学


经济


物理


其他领域


用户生成数据


Deep Web数据



多模态内容数据



网络与关系数据

11.1 大数据

➢大数据计算模式

表13-1 大数据计算模式及其代表产品

大数据计算模式	解决问题	代表产品
批处理计算	针对大规模数据的批量处理	MapReduce、Spark等
流计算	针对流数据的实时计算	Storm、S4、Flume、Streams、Puma、DStream、Super Mario、银河流数据处理平台等
图计算	针对大规模图结构数据的处理	Pregel、GraphX、Giraph、PowerGraph、Hama、GoldenOrb等
查询分析计算	大规模数据的存储管理和查询分析	Dremel、Hive、Cassandra、Impala等

11.1 大数据

➢大数据管理的基础—数据中心

- 数据中心—看上去，规模庞大



脸谱(Facebook) 数据中心



微软(Microsoft)数据中心



苹果(Apple) 数据中心



谷歌(Google) 数据中心

11.1 大数据

➢大数据管理的基础—数据中心

- 数据中心—走进去，结构复杂

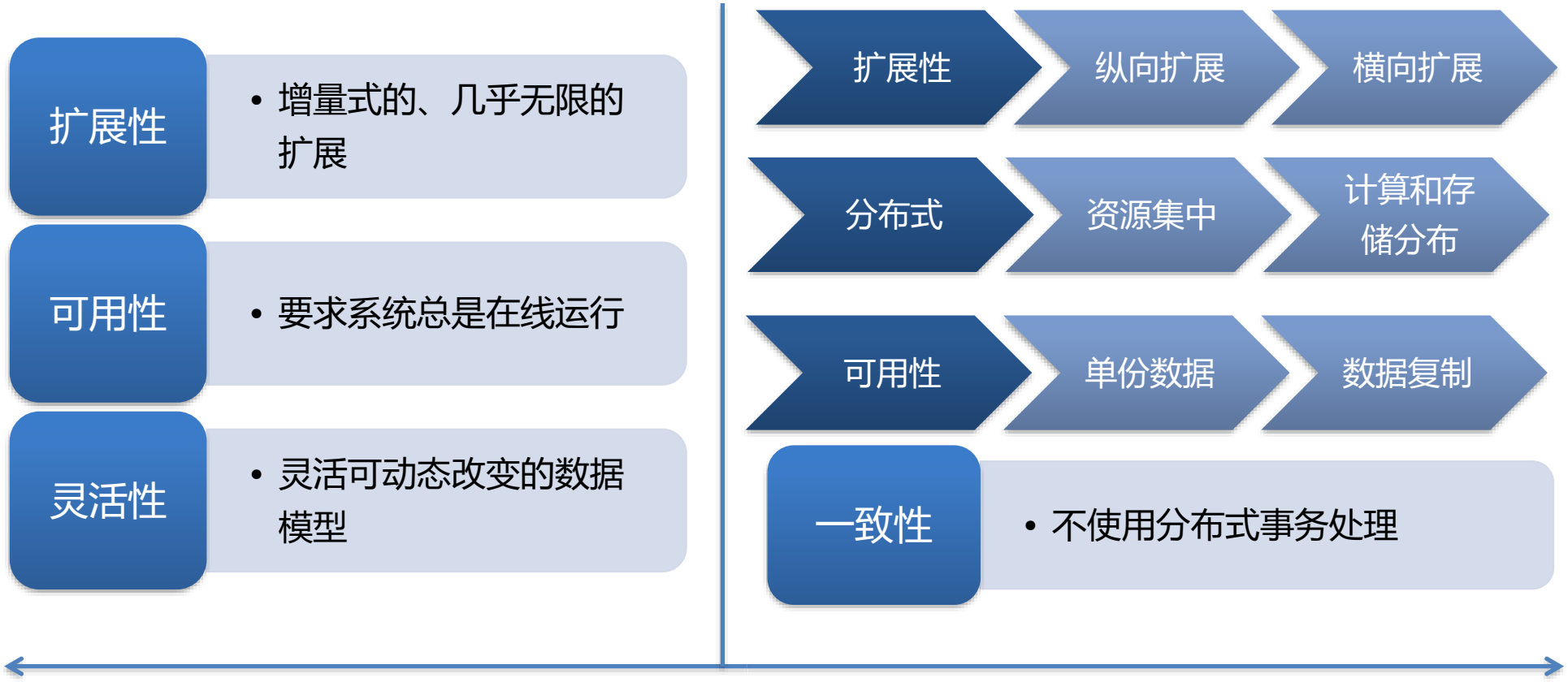






11.1 大数据

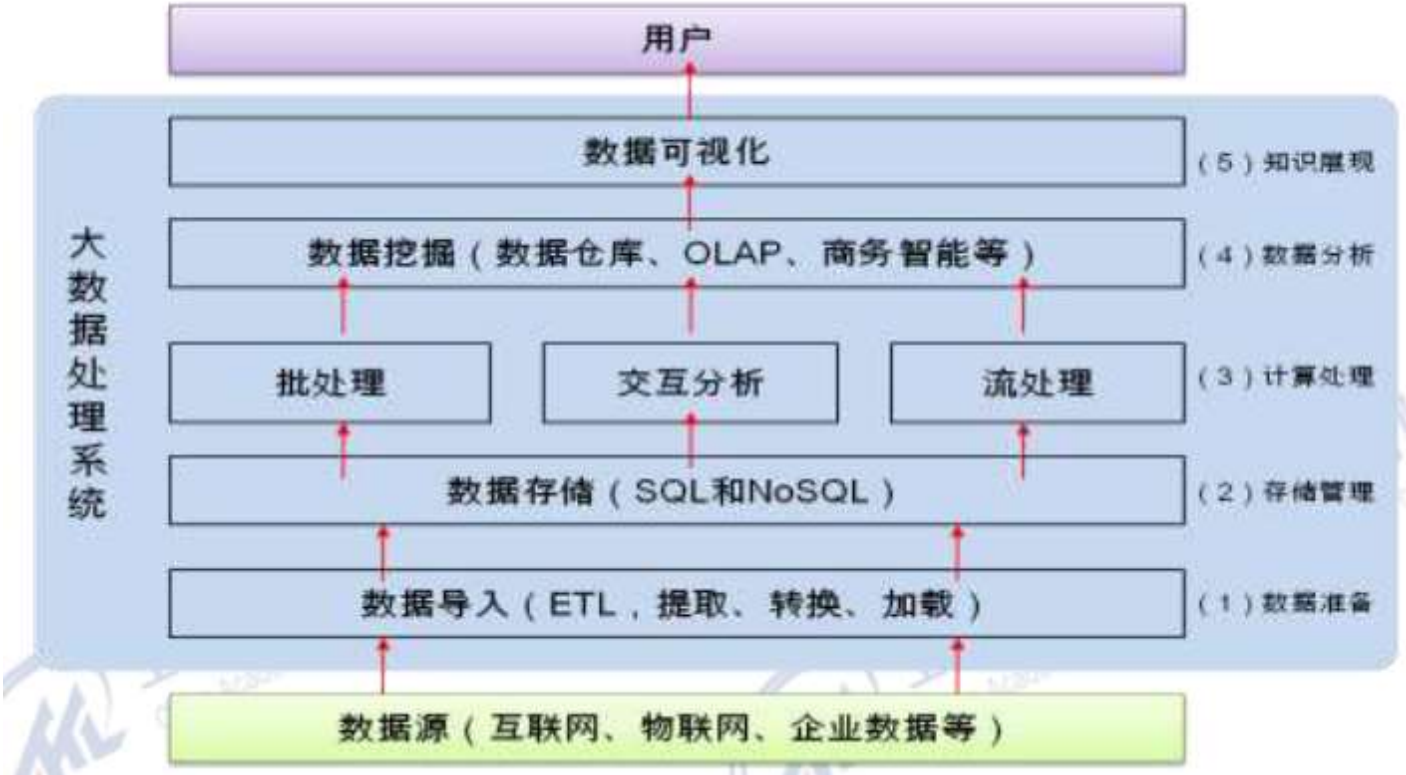
➢大数据处理的需求和特点



11.1 大数据

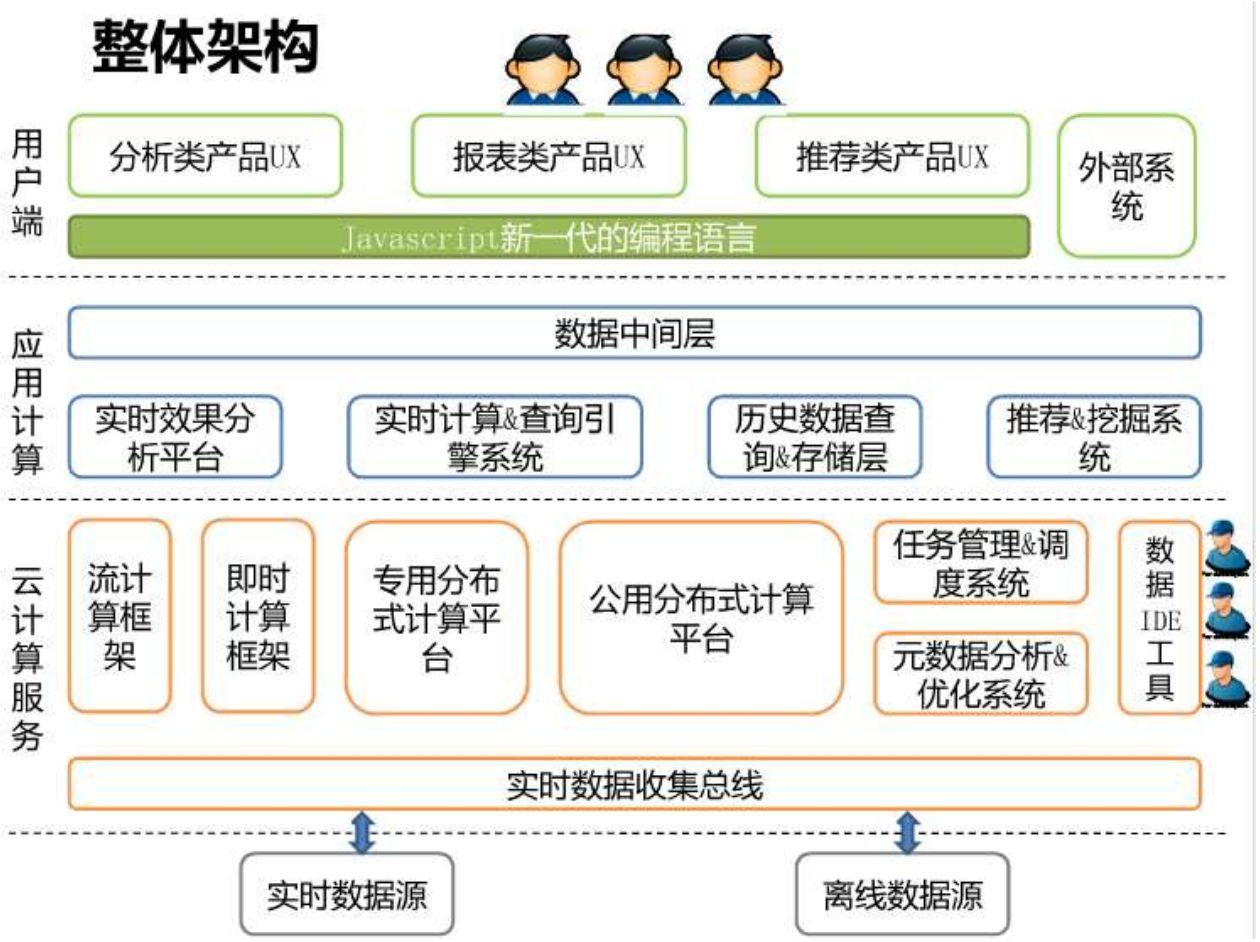
➢大数据架构：分层架构

- 从数据在生命周期看，大数据从数据源经过分析挖掘到最终获得价值需要经过5个环节，包括数据准备、数据存储与管理、计算处理、数据分析和知识展现。



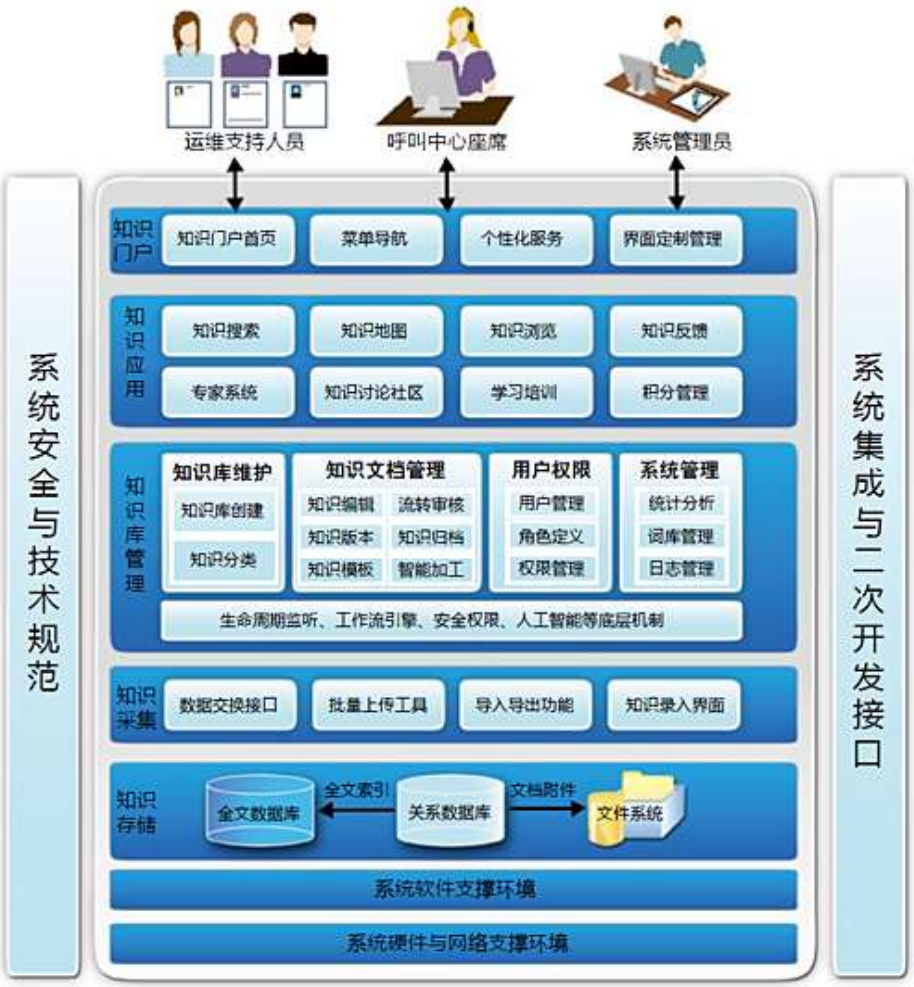
11.1 大数据

➢大数据的系统架构：整体系统架构



11.1 大数据

➢大数据架构：整体逻辑功能架构



11.2 云计算

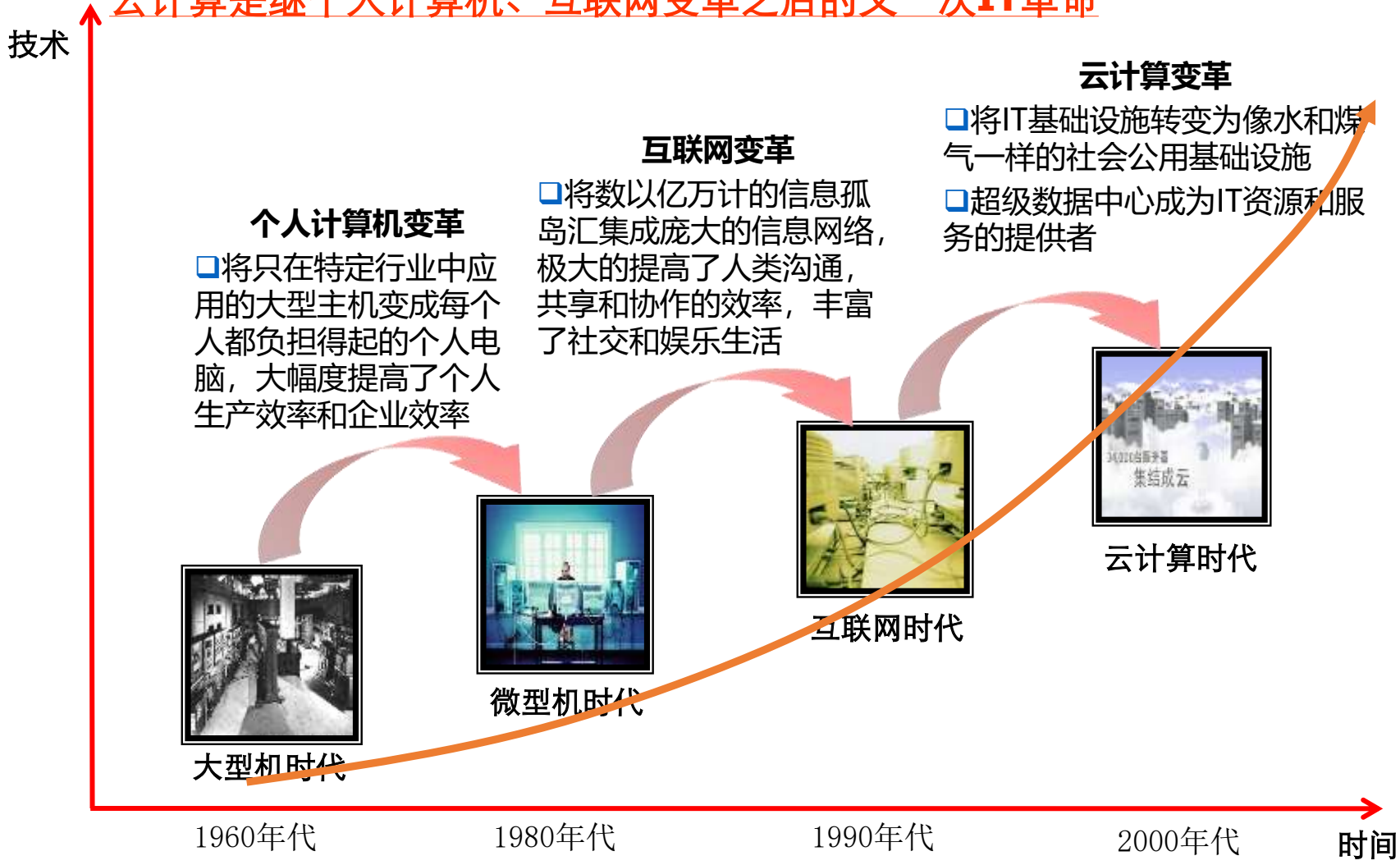
➢什么是云计算?

- 云计算是一种商业计算模型。它将计算任务分布在大量计算机构成的资源池上，使各种应用系统能够根据需要获取计算力、存储空间和信息服务。
- 运用由网络连接起来的大量虚拟化的计算设备对信息进行处理并得到结果的一种计算方式。
- 云计算不是一种产品，也不是一种技术，而是一种产生和获取计算能力的运营模式。

11.2 云计算

➢云计算的由来

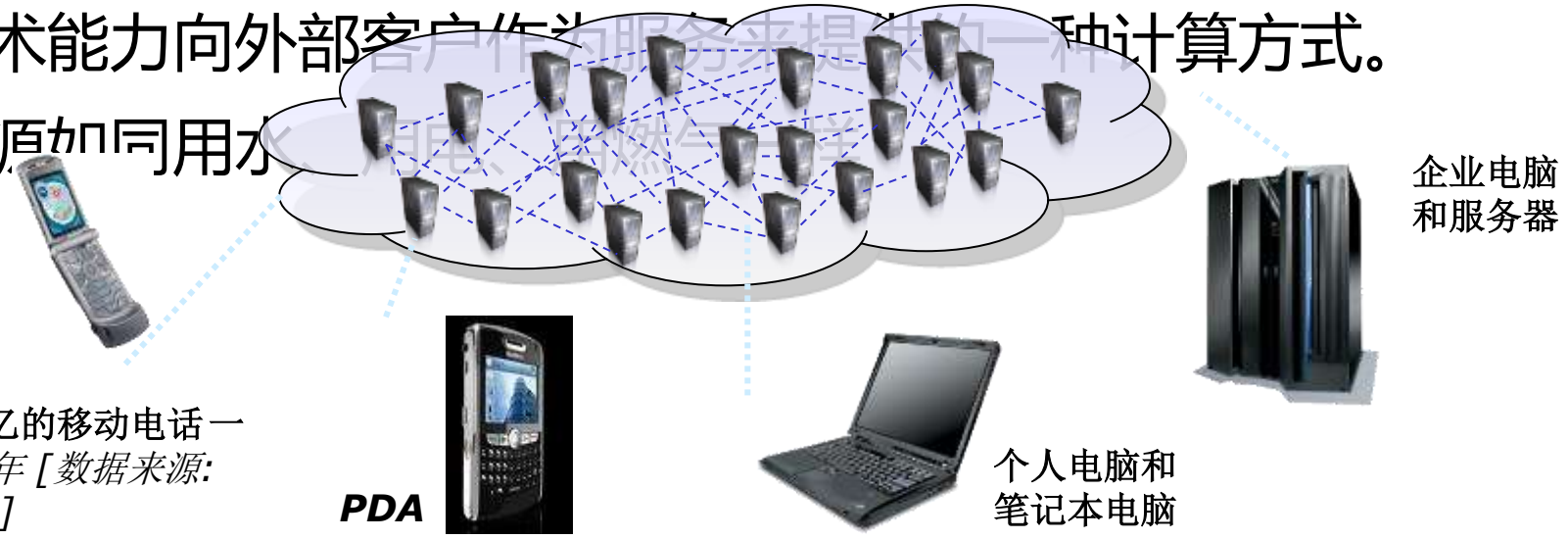
云计算是继个人计算机、互联网变革之后的又一次IT革命



11.2 云计算

➢云计算定义

- 云计算(Cloud Computing)是分布式处理(Distributed Computing)、并行处理(Parallel Computing)和网格计算(Grid Computing)的发展，或者说是这些计算机科学概念的商业实现。
- 云计算是指基于互联网的超级计算模式--即把存储于个人电脑、移动电话和其他设备上的大量信息和处理器资源集中在一起，协同工作。在极大规模上可扩展的信息技术能力向外部客户作为服务来提供的一种计算方式。
- 使得计算资源如同用水、用电、煤气一样



11.2 云计算





计算起源和发展

- 1961年斯坦福教授John McCarthy 提出计算资源可以成为一种重要的新型工业，类似水、电、气和通信。
- 1999年Salesforce成立，2001年发布在线CRM系统。
- 2006年Google CEO Eric Schmidt 在搜索引擎大会上首次提出”Cloud Computing“概念。
- 2006年Google逐步开始在内部使用云计算，2008年推出Google AppEngine云计算平台。
- 2006年Amazon正式对外推出弹性计算服务(EC2) 。
- …各大全球知名厂商跟进(IBM,MicroSoft….) 。

11.2 云计算


云计算
并行计算
分布式计算
网格计算
●同时使用多种计算资源解决计算问题的过程
●将需要巨大计算能力的问题分成许多小部分进行处理，最后综合结果
●在动态、多机构参与的虚拟组织中协同共享资源和求解问题

11.2 云计算

➢从不同角度看云计算的内涵

- 云计算是一种IT基础设施交付和使用模式
  - 从原理看，云计算是将计算工作分布在大量的、分布式的计算机上，从而利用大量计算资源(包括硬件设备、软件及网络等IT基础设施)完成所要求的计算任务。

- 云计算是一种信息服务的交付和使用模式
  - 从商业层面看，云计算提供的是而且仅是服务——可以统称为“云服务”。
  - 从云计算服务商和云计算用户间的关系看，服务商并未向用户提供硬件、软件等事物或具体产品，而是运用自己所掌握的IT资源帮助用户完成计算任务，满足用户使用软硬件设备或获得信息的需要。

11.2 云计算

➢从不同角度看云计算的内涵

- 云计算是一种基于互联网的新型计算模式
  - 从数值和信息处理的角度看，云计算是通过互联网将计算任务分布到由大量计算机构成的资源池上，从而使用户能够根据需要获取计算能力、存储空和信息服务。
  - 云计算的四方面新特征：
    - 资源汇聚：计算资源汇聚在一起通过多租户模式服务多个消费者
    - 快速弹性：计算能力能够以某种自动方式快速而弹性地实现供应
    - 按需的自助服务：消费者能够根据自己的需要获取计算能力
    - 4.可控可计量：系统根据服务类型提供相应的计量方式

11.2 云计算

➢云服务

云计算服务对象：在各行各业、各种组织、各种产业拓展应用范围

从行业视角
- 满足中小企业、政府部门、科研教育、娱乐、商业企业的信息化需求
- 成为互联网企业的运营平台和资源平台
- 成为人民群众生活的必需品

从产业领域视角
- 成为物联网信息存储、分析的智能化平台
- 成为移动互联网的基础信息支撑平台
- 成为移动电子商务、移动支付的支持平台
- 逐步进入网络通信基础设施领域

云计算服务内容：从基础架构、在线存储、软件服务扩展到各种应用领域

已有主要服务内容
- **基础设施**即服务/**I**aaS
- **软件平台**即服务/**P**aaS
- **软件**即服务/**S**aaS
- **存储**即服务/**D**aaS

XaaS

一切皆服务，延伸到各种应用
- **硬件**即服务：终端设备成为服务发布形式
- **消息**即服务：消息中间件成为服务
- **监控**即服务：监控内容成为服务
- 现存各种应用均被**云服务化**...

云计算服务形式：更简单易用、更安全、更智能的差异化服务

- 更为**便捷**：就像现在水、电、气、通信那样简单，打开即用，关闭即停，按需使用，用完充值
- 隐私和信息**安全**更有保障：用户信息隔离、网络传输加密、高强度身份认证、服务商数据安全审核
- 更**智能**化：用户服务状态恢复、信息推荐、精准营销、预测用户潜在信息需求、
- **差异化**服务：提供从免费、普通用户、中端用户、高端用户的各种服务，按质论价

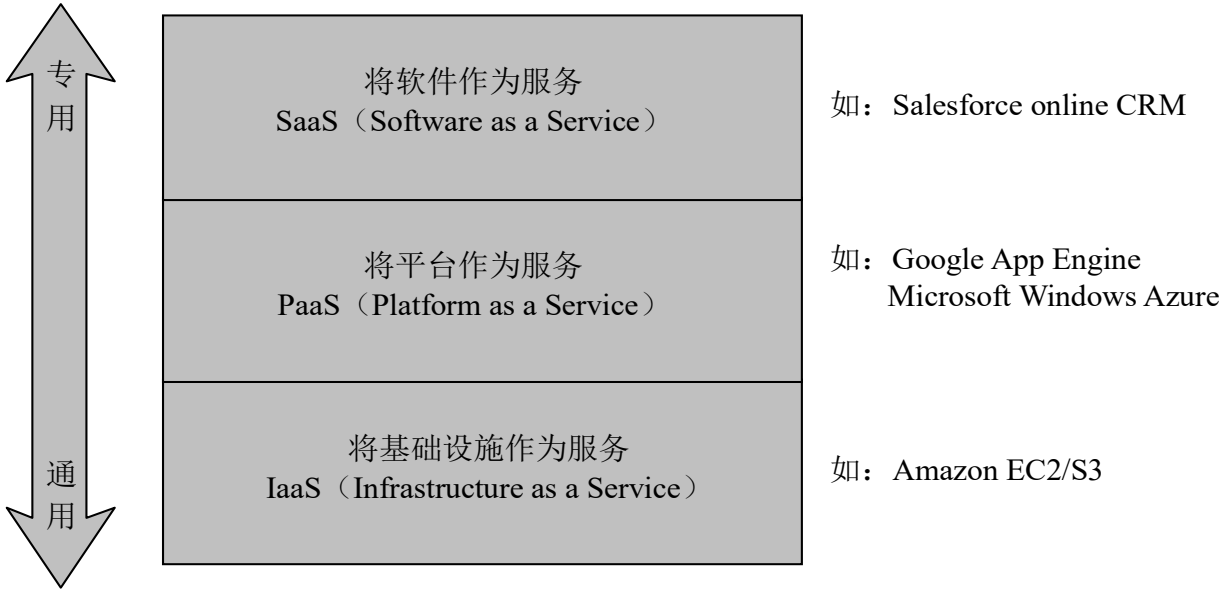
11.2 云计算

➢云计算特点

- 超大规模
- 虚拟化
- 高可靠性
- 高通用性
- 高可伸缩性
- 多租户
- 按需服务
- 极其廉价

11.2 云计算

➢云计算服务类型

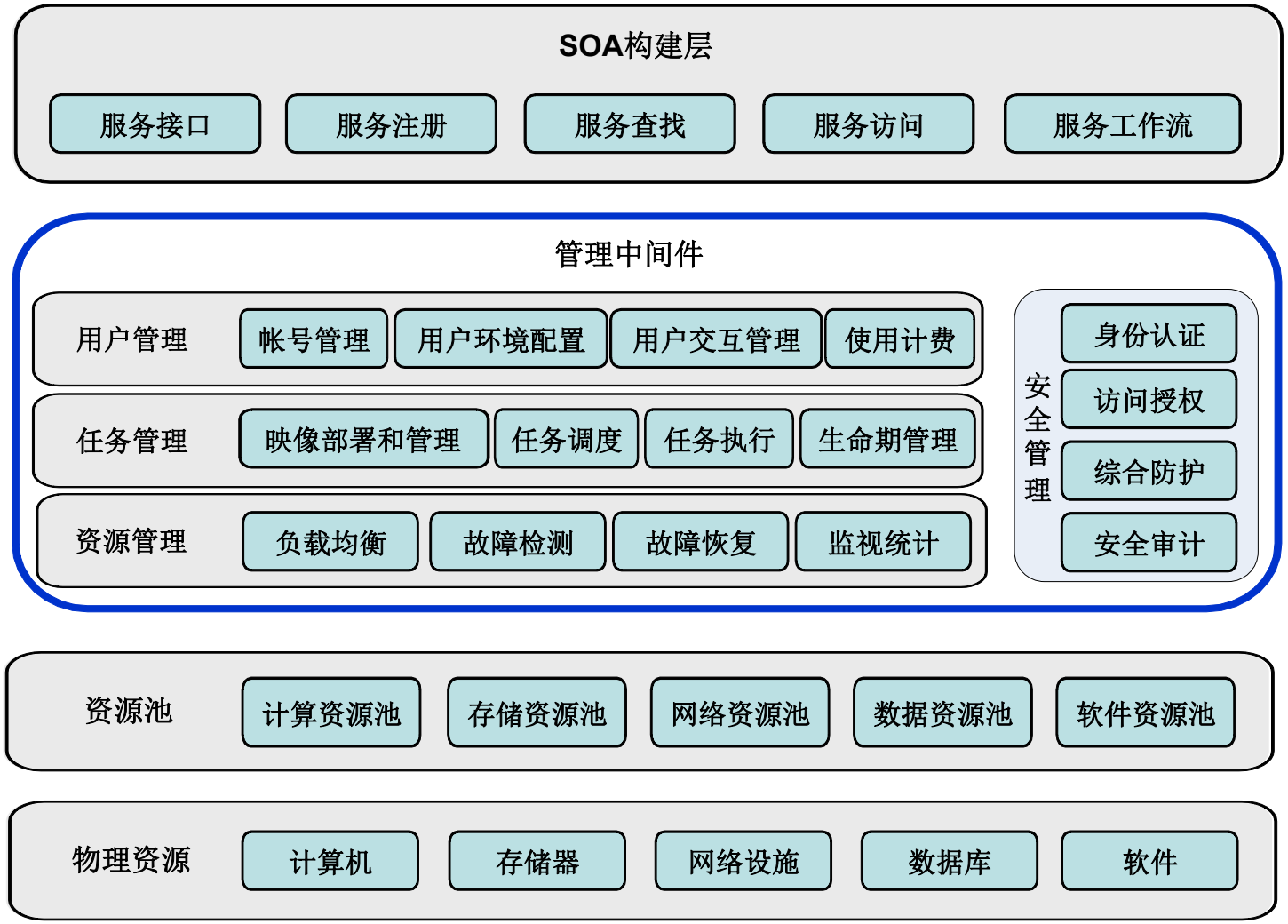




- 将硬件设备等基础资源封装成服务供用户使用
- 对资源的抽象层次更进一步，提供用户应用程序运行环境
- 针对性更强，它将某些特定应用软件功能封装成服务

11.2 云计算

➢云计算技术体系结构



11.2 云计算

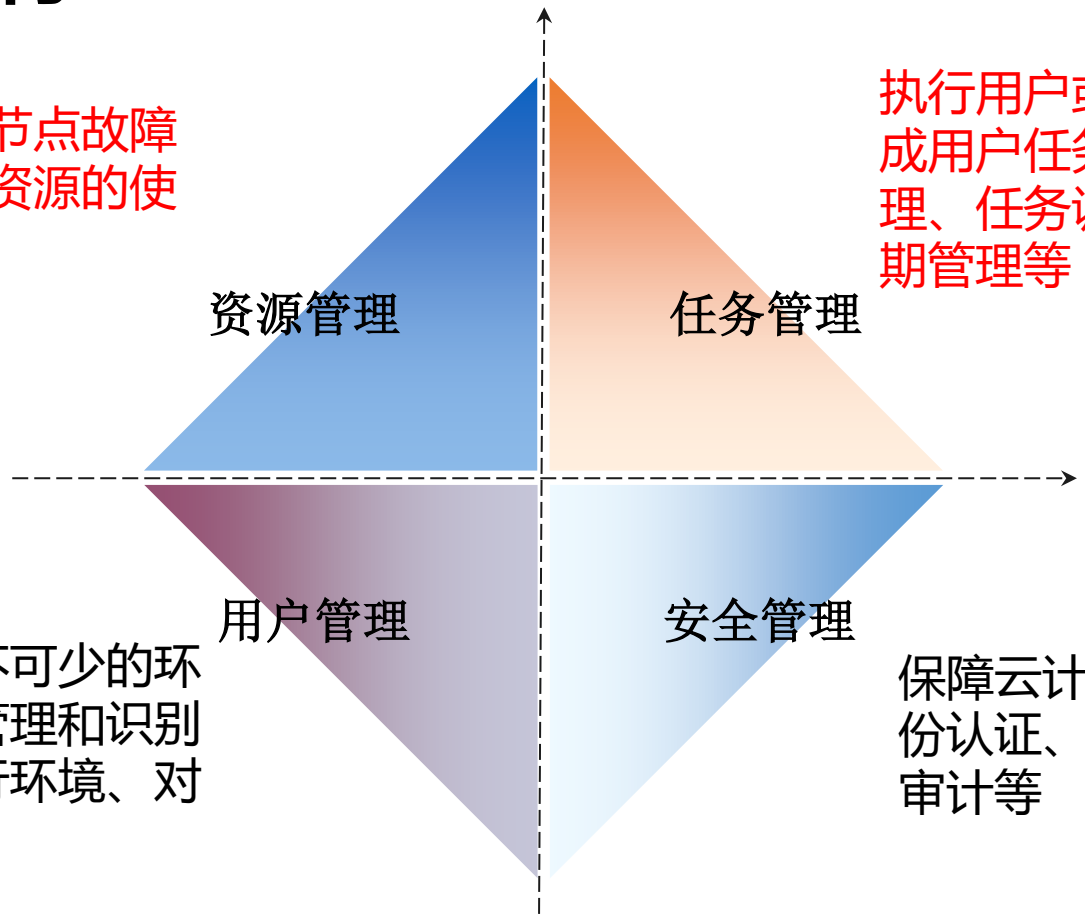
➢云计算技术体系结构



11.2 云计算

➢云计算技术体系结构

均衡使用云资源节点，检测节点故障并试图恢复或屏蔽之，并对资源的使用情况进行监视统计



执行用户或应用提交的任务，包括完成用户任务映象(Image)的部署和管理、任务调度、任务执行、任务生命期管理等

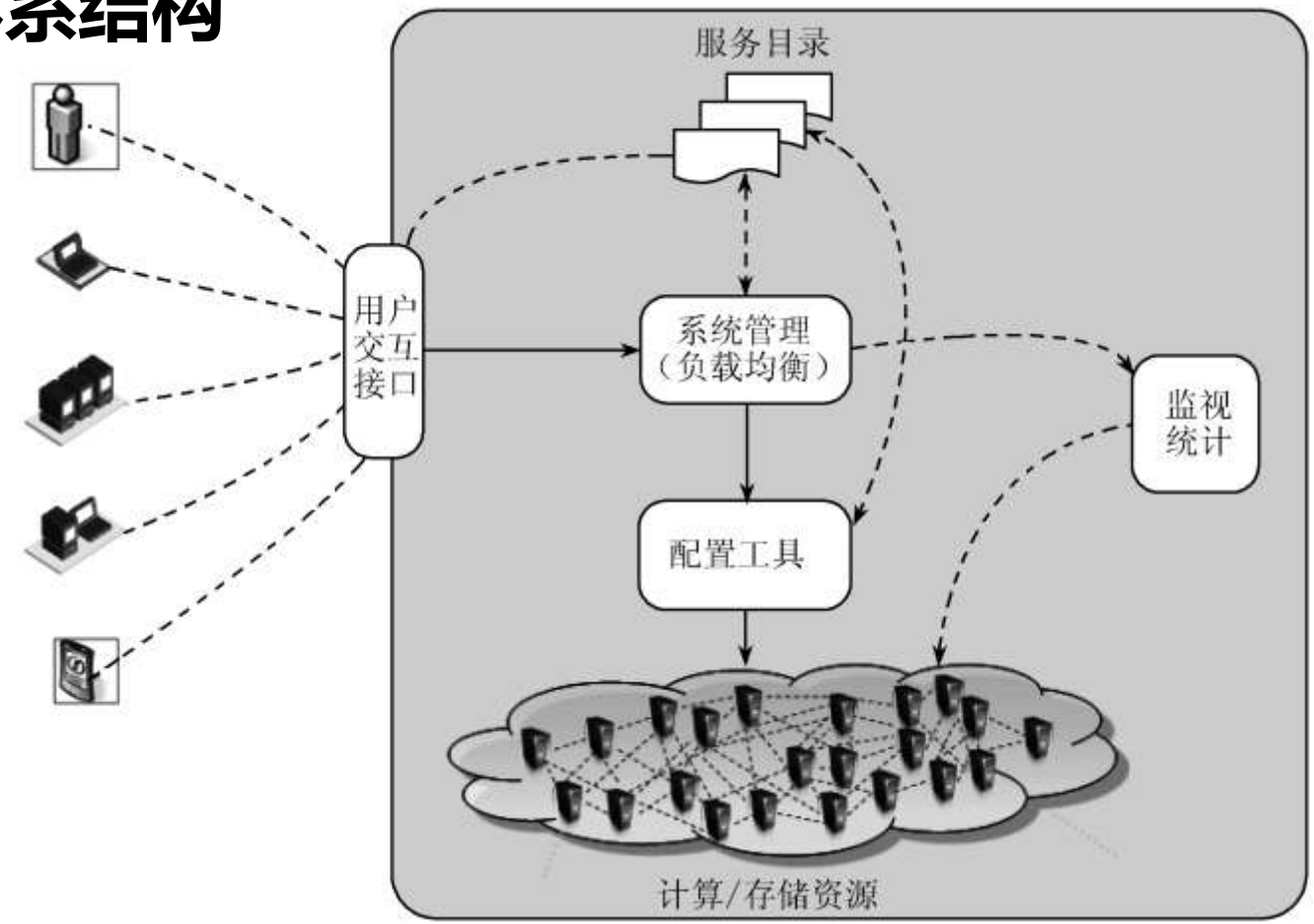
实现云计算商业模式的一个必不可少的环节，包括提供用户交互接口、管理和识别用户身份、创建用户程序的执行环境、对用户的使用进行计费等

保障云计算设施的整体安全，包括身份认证、访问授权、综合防护和安全审计等

云计算的管理中间件层

11.2 云计算

➢云计算技术体系结构



简化的IaaS实现机制图

11.2 云计算

➢云计算的优势

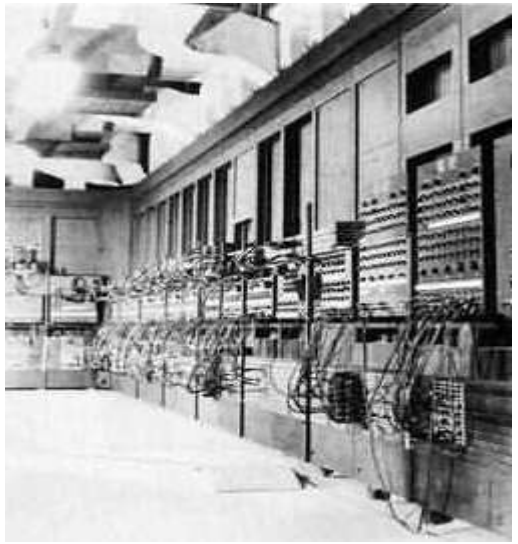
- 数据在云端：不怕丢失,不必备份,可以任意点的恢复；
- 软件在云端：不必下载自动升级；
- 无所不在的计算：在任何时间地点，任何设备登录后就可以进行计算服务；
- 无限强大的计算：具有无限空间的、无限速度。

PC	C/S	云计算
		
硬件为中心	软件为中心	服务为中心

11.2 云计算

➢天下大势，合久必分，分久必合，计算机技术的分合演义

- 早期计算技术以“合”为特征:曲高和寡
- PC的发展使“分”成为了主流:
  - 旧时王谢堂前燕，飞入寻常百姓家
- 网络技术的发展使云计算成为了“合”的模式，计算和存储通过网络隐于云端:
  - 大象无形







11.2 云计算

➢云计算体系结构

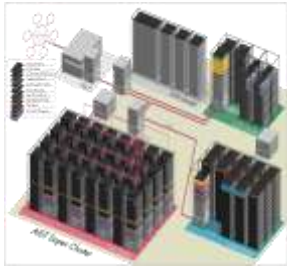
- 云计算的基本原理是通过使计算分布在大量的分布式计算机上，而非本地计算机或远程服务器中，企业数据中心的运行将更与互联网相似。
- 这使得企业能够将资源切换到需要的应用上，根据需求访问计算机和存储系统。

11.2 云计算

➢云计算发展路线



并行计算



集群计算



网格计算

云计算



11.3 云数据管理

➢什么是云数据管理?

- 目前还没有统一定义!
- 一般认为，云数据管理是指对存储在云计算环境下的海量数据资源进行有效组织管理、运行维护的大型分布式数据管理体系架构。
- 从面向服务的角度看，云数据管理又可称为DaaS(Data as a Service，数据即服务)，即以新一代大规模分布式云存储为基础的数据服务系统，其核心是以No SQL(New SQL)为代表的新一代云数据存储(库)系统。

11.3 云数据管理

➢云数据管理的背景

- 随着互联网的日益普及和IT 技术的迅猛发展，互联网上的数据急剧膨胀，如何存储和管理海量数据已成为一个亟待解决的挑战性问题。
- 为此，云计算的概念应运而生，并彻底改变了数据存储的基础架构。

11.3 云数据管理

➢云数据管理的背景

- Web应用作为推动云计算发展的主力，正快速的普及成熟起来，它需要能够按需进行扩展，企业都希望他们的Web应用系统能够应对不断增长的用户量和数据量，以及适应用户和其它开发人员对WEB应用系统的高吞吐量和低响应时间的要求。
- 系统必须以低延迟的响应速度(几十毫秒)、高吞吐量(每秒上万的读写)为世界范围内的用户请求提供服务。
- 并且，应用必须具备高可用性，长期维持最小的运维成本。
- 但是，如果WEB应用系统仍然依赖传统的关系(SQL)数据库系统，我们会发现数据存储将成为最大的瓶颈。

11.3 云数据管理

➢云数据的特点

- 海量性
- 异构性
- 不确定性
- 容错性

11.3 云数据管理

➢传统关系型数据库的限制

- 扩展困难
- 读写速度慢
- 成本高
- 有限的支撑容量

11.3 云数据管理

➢云数据管理的需求

- 低延迟的读写速度
- 支撑海量的数据和流量
- 大规模集群的管理
- 庞大运营成本的考量

11.3 云数据管理

➢云数据管理准则(1)

- 切分万物以治之
- Partition Everything and key-value storage
- 数据库的第一范式无法满足

11.3 云数据管理

➢云数据管理准则(2)

- 容不同乃成大同
- Embrace Inconsistency
- 数据库的ACID属性无法满足。

11.3 云数据管理

➢云数据管理准则(3)

- 狡兔三窟方高枕
- Backup everything with three copies
- 每个数据都有三个备份，达到 99.999%的可用性。

11.3 云数据管理

➢云数据管理准则(4)

- 运筹沧海量兼容
- Scalable and high performance
- 提供一个可伸缩的，处理和分析海量数据的平台。

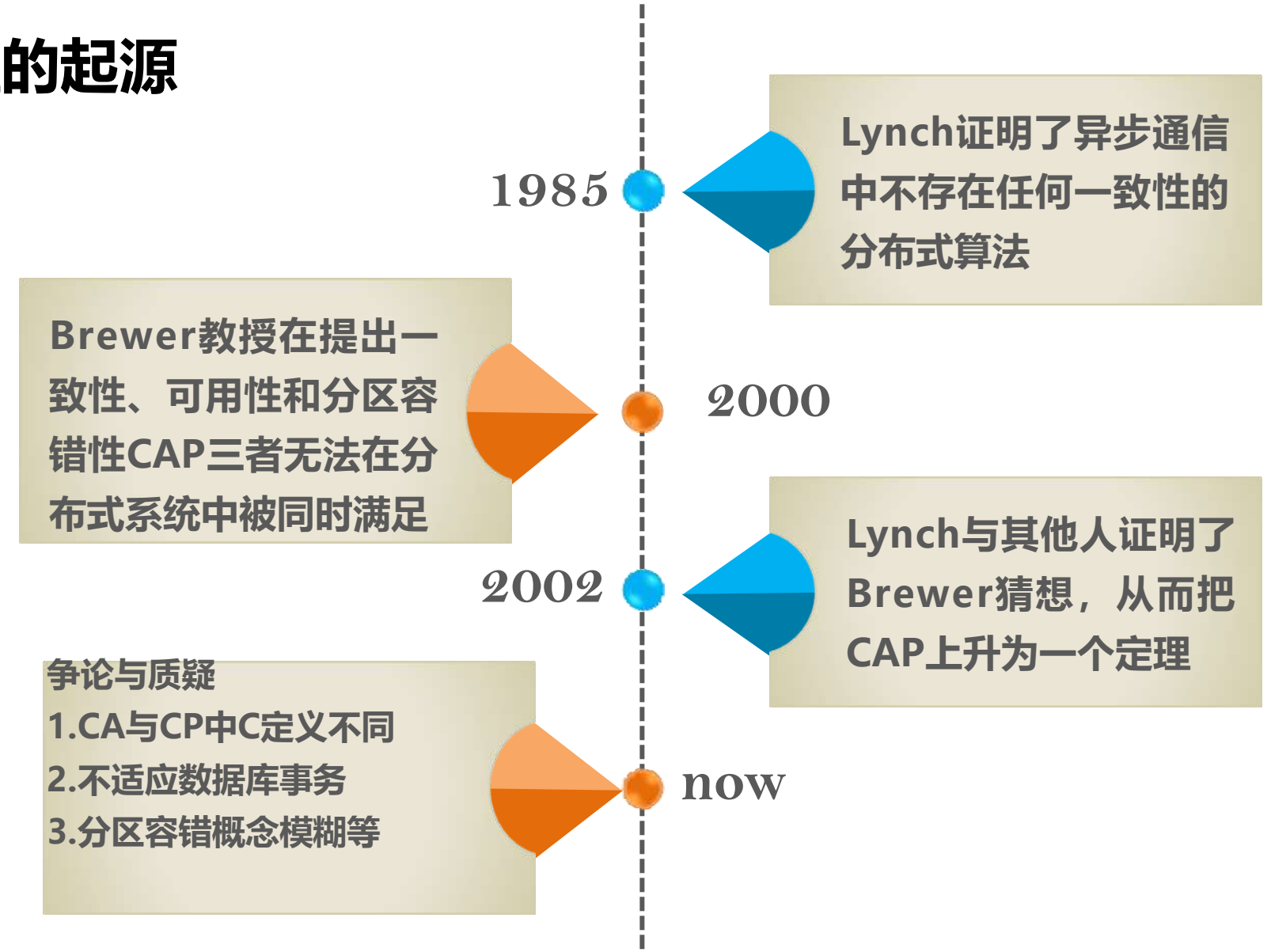
11.3 云数据管理

➢云数据管理的理论基础

- CAP定理
- BASE特性
- 一致性哈希
- PAXOS算法

11.3 云数据管理

➢CAP定理的起源



11.3 云数据管理

➢CAP定理：描述

- 一致性(Consistency)：在分布式系统中的所有 有同样的值。(等同于所有节点访问同一份最新的
- 可用性(Availability)：在集群中一部分节点故障 户端的读写请求。(每一个操作总是能够在一定
- 分区容忍性(Partition tolerance)：允许节点之 分区发生时，节点之间的消息可能会完全丢失



➢一个分布式系统不可能同时满足一致性，可 需求，三个要素中最多只能同时满足两点。三者不可兼顾

11.3 云数据管理

➢CAP定理

- CAP定理(Brewer's CAP Theorem)是云数据管理的理论基础之一。
- 它最初是由 Eric Brewer教授(University of California，Berkeley)提出的有关分布式数据共享系统性质一个猜想, 后来Seth Gilbert 和 Nancy Lynch证明了其正确性, 因此成为定理。

11.3 云数据管理

➢CAP定理

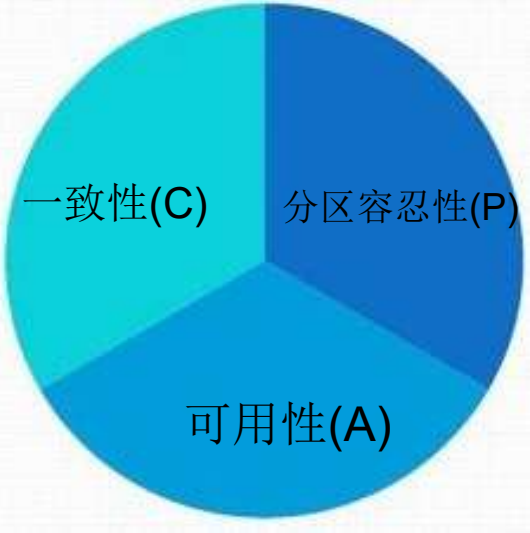
- 一个分布式系统有三个最重要的性质:
  - 一致性(C)
  - 可用性(A)
  - 分区容忍性(P)
    - Consistency (all copies have same value)
    - Availability (system can run even if parts have failed)
    - Partitions (network can break into two or more parts, each with active systems that can’ t talk to other parts)



11.3 云数据管理

➢CAP定理

- CAP可以同时得到吗?
- 鱼与熊掌可以兼得吗?
- 答案：NO!
- 一个分布式系统最多只能同时满足两个
- Brewer’s CAP “Theorem”: You can have at most two of these three properties for any system





11.3 云数据管理

➢CAP定理

- CAP定理告诉我们:在设计分布式数据共享系统时, 熊掌与鱼不可兼得也!
- 因此,架构师不要将精力浪费在设计能满足三者的完美分布式系统，而是应该进行取舍。
- 例如, 如果关注的是一致性(C)，那么就需要处理因为系统不可用而导致的写操作失败的情况;
- 而如果关注的是可用性(A)，那么就应该知道系统的read操作可能不能精确的读取到write操作写入的最新值。

11.3 云数据管理

➢CAP定理

- 由于系统的关注点不同，相应的采用的策略也是不一样的，只有真正的理解了系统的需求，才有可能利用好 CAP理论。
  - CA：传统关系数据库
  - AP：key-value数据库

11.3 云数据管理

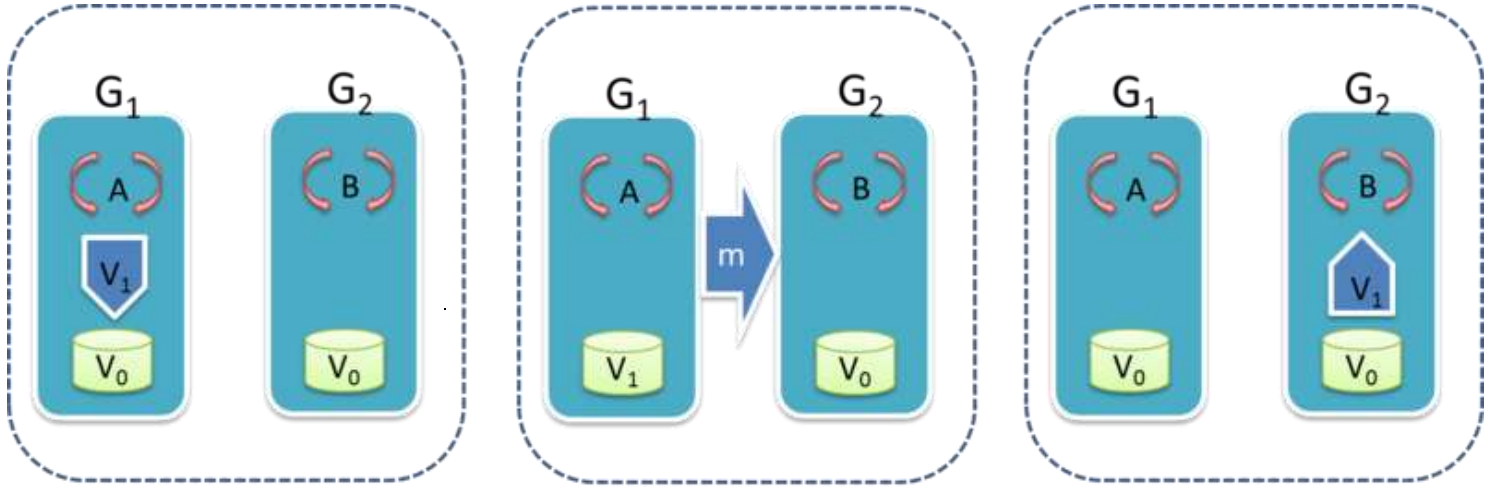
➢CAP定理

- 应用CAP定理时必须注意不同数据对于一致性的要求是不同的。
- 举例来讲，用户评论对不一致是不敏感的，可以容忍相对较长时间的不一致，这种不一致并不会影响交易和用户体验。
- 而产品价格数据则是非常敏感的，通常不能容忍超过10秒的价格不一致。

11.3 云数据管理

➢CAP定理:示例

- 在正常情况下，操作过程如下(如下图所示)：
  - A将V0更新为V1
  - G1发送消息m给G2，数据V0更新为V1
  - B读取到G2中的数据V1

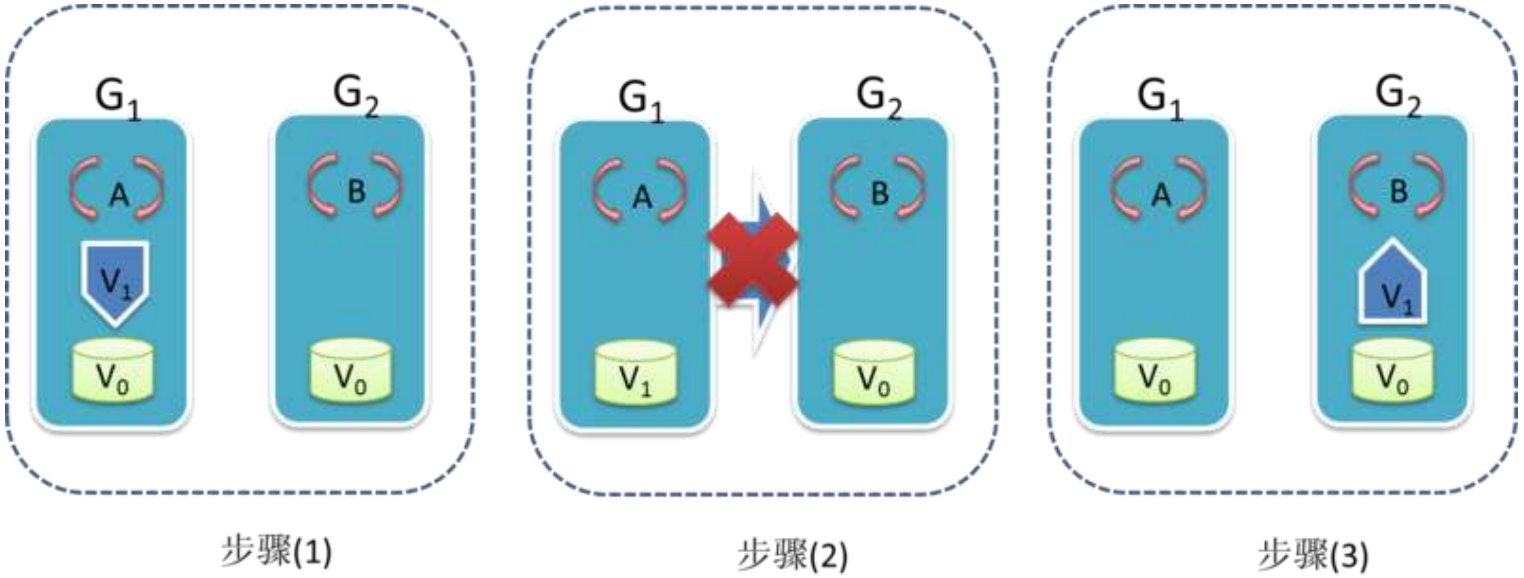


步骤(1)　步骤(2)　步骤(3)

11.3 云数据管理

➢CAP定理:示例

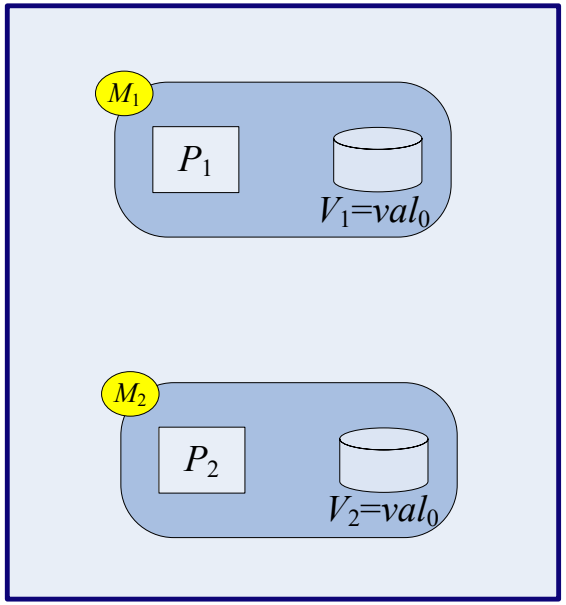
- 发生网络分区故障，G1发送的消息不能传送到G2上。
- 这样数据就处于不一致的状态，B读取到的就不是最新的数据。
- 如果采用阻塞，加锁等机制来保证数据一致性，就会影响系统可用性。



11.3 云数据管理

➢CAP定理:示例

- 一个牺牲一致性来换取可用性的实例

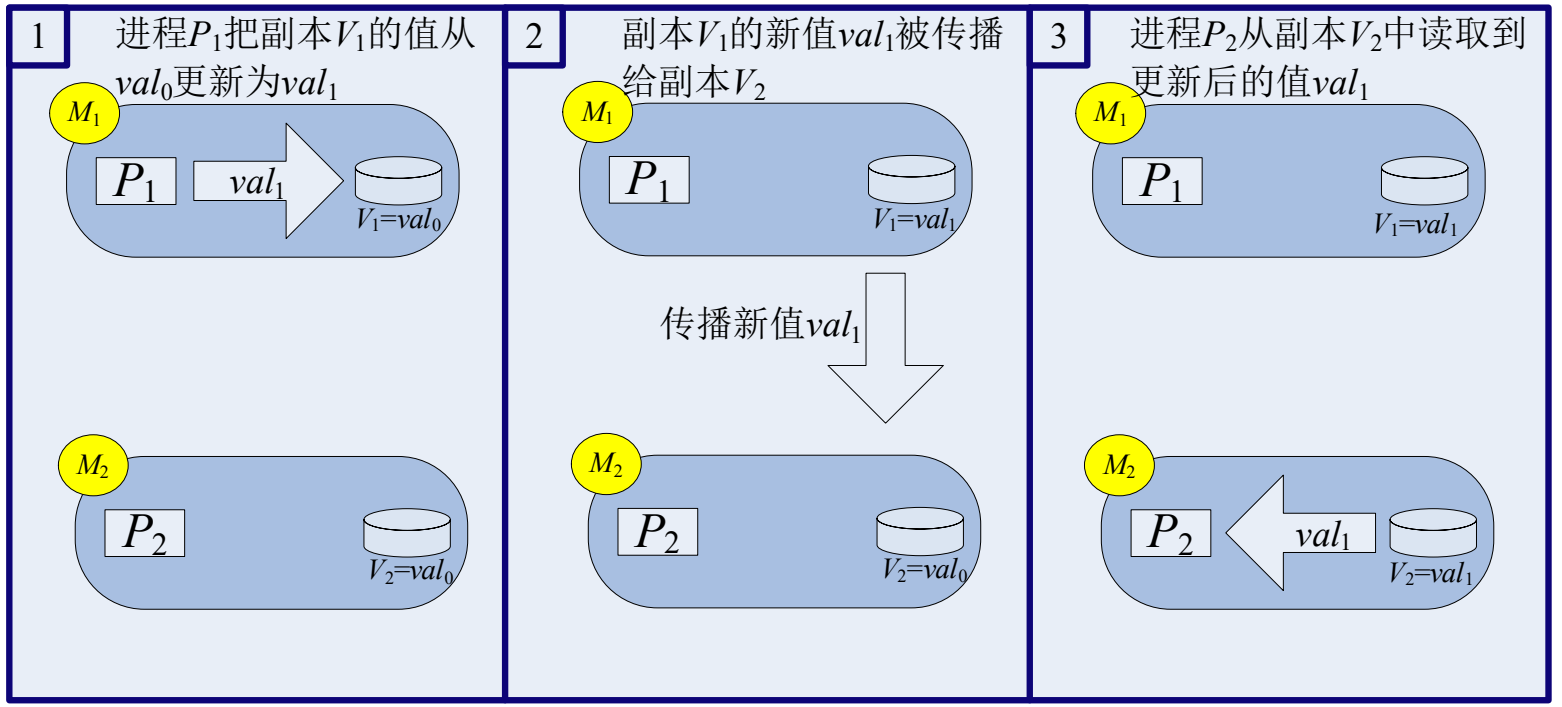


(a)初始状态

11.3 云数据管理

➢CAP定理:示例

- 一个牺牲一致性来换取可用性的实例

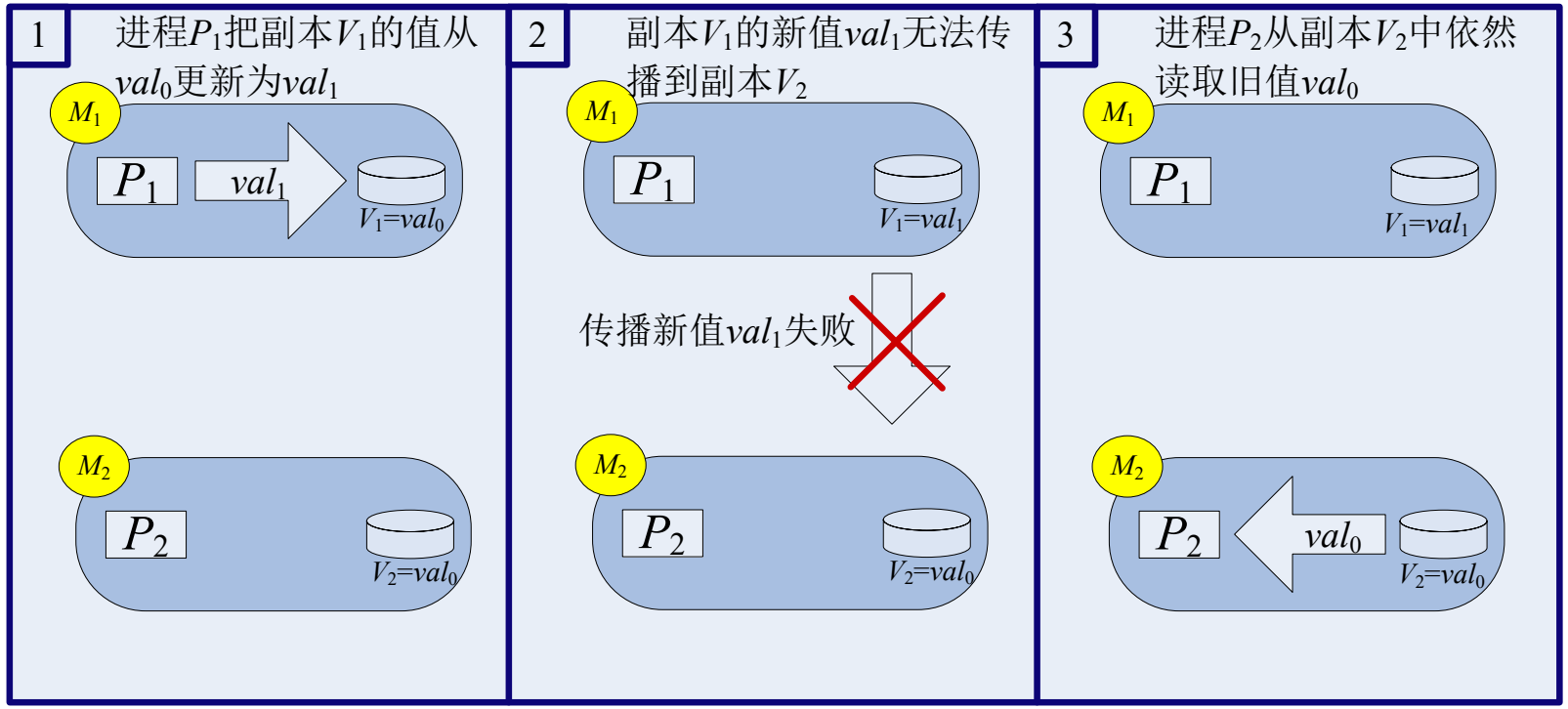


(b)正常执行过程

11.3 云数据管理

➢CAP定理:示例

- 一个牺牲一致性来换取可用性的实例



(c) 更新传播失败时的执行过程

11.3 云数据管理

➢CAP定理：选择放弃

- 放弃P：如果想避免分区容错性问题的发生，一种做法是将所有的数据(与事务相关的)都放到一台机器上。虽然无法100%地保证系统不会出错，但不会碰到由分区带来的负面效果。当然，这个选择会严重影响系统的扩展性。
- 放弃A：一旦遇到分区容错故障，那么受到影响的服务需要等待数据一致，因此在等待期间系统无法对外提供服务。
- 放弃C：这里所说的放弃一致性，并不是完全放弃数据的一致性，而是放弃数据的强一致性，而保留数据的最终一致性。以网络购物为例，对只剩最后一件库存的商品，如果同时接收到了两份订单，那么较晚的订单将被告知商品售罄。

11.3 云数据管理

➢CAP定理：选择放弃

- 为什么选择 AP，放弃C
  - 小的延迟(数据库达到一致的状态)不足以对总体性能或用户体验造成影响。
  - 不好的用户体验，带来的影响Amazon 每0.1秒的响应延迟都会导致1%的销售降低。Google 0.5秒的延迟会使流量减少15%。

- 其他选择
  - 引入BASE(Basically Available, Soft-state, Eventually consistent)，该方法支持最终一致性，其实是放弃C的一个特例

11.3 云数据管理

➢CAP定理：选择放弃

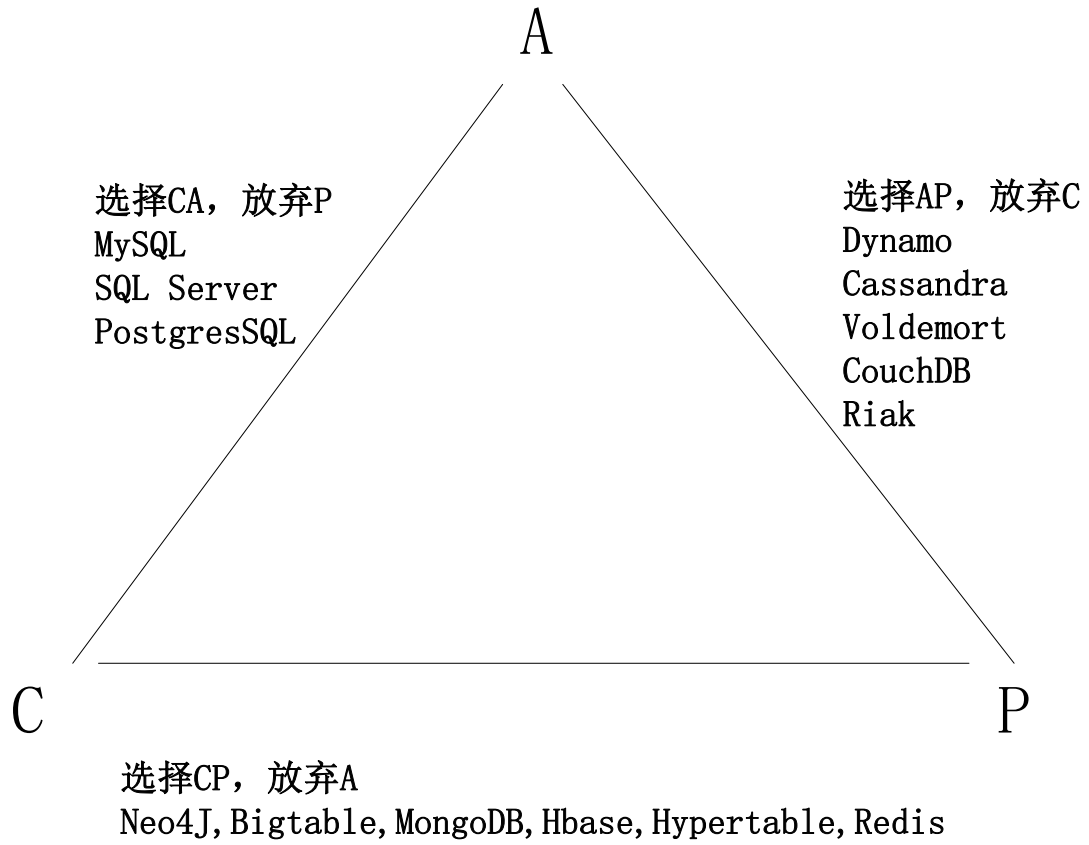


图 不同产品在CAP理论下的不同设计原则

11.3 云数据管理

➢一致性模型--背景

- 当我们在生产线上用一台服务器来提供数据服务的时候，会遇到如下两个问题：
  - 一台服务器的性能不足以提供足够的能力服务所有的网络请求。
  - 我们总是担心这台服务器停机，造成服务不可用或是数据丢失。
- 通常，我们会通过两种手段来扩展数据服务：
  - 数据分区：就是把数据分块放在不同的服务器上(如：uid % 16，一致性哈希等).
  - 数据镜像：让所有的服务器都有相同的数据，提供相当的服务。
- 那么问题来了：
  - 在数据分区的方案中：如何支持分布式事务(Two-Phrase Commit)
  - 数据镜像方案中：如何保证数据的一致性

11.3 云数据管理

➢一致性模型--背景

- 我们只讨论在数据冗余情况下考虑数据的一致性和性能的问题。简单说来：
  - 要想让数据有高可用性，就得写多份数据。
  - 写多份的问题会导致数据一致性的问题。
  - 数据一致性的问题又会引发性能问题。

11.3 云数据管理

➢数据一致性--分类

- 强一致性
  - 强一致性(即时一致性)假如A先写入一个值到存储系统，存储系统保证后续的读操作都返回最新值。
  - 例如：文件系统，RDBMS都是强一致性的。
- 弱一致性
  - 假如A先写入了一个值到存储系统，存储系统不能保证后续读取操作能够读到最新值。
  - 这种情况下有一个“不一致性窗口”，它特指从A写入值，到后续操作A,B,C读取到最新值这一段时间。
  - 例如：某些cache系统，网络游戏其它玩家的数据和你没什么关系，或是百度搜索引擎。

11.3 云数据管理

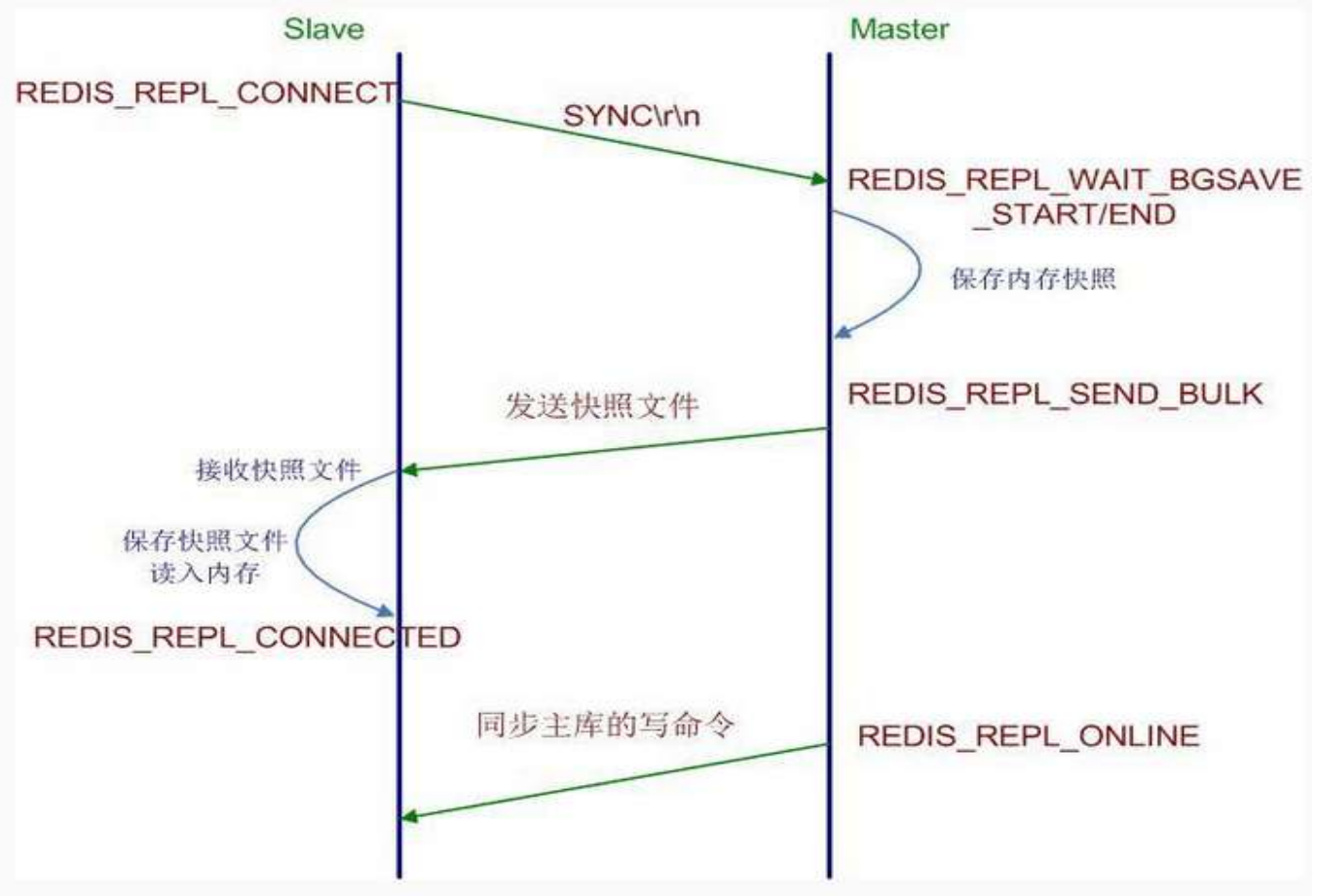
➢数据一致性--分类

- 最终一致性
  - 最终一致性是弱一致性的一种特例
  - 假如A首先write了一个值到存储系统，存储系统保证，如果在A,B,C后续读取之前没有其他写操作更新同样的值的话，最终所有的读取操作都会读取到A写入的最新的值。
  - 这种情况下，如果没有失败发生的话，“不一致性窗口”的大小依赖以下的几个因素：
    - 交互延迟
    - 系统的负载
    - 复制架构中replica的个数(可以理解为master/slave模式中，slave的个数)
  - 例如：DNS，电子邮件。

11.3 云数据管理

➢一致性模型--最终一致性

- Master-Slave(redis):
  - 读写请求都由Master负责。
  - 写请求写到Master上，然后由Mas



11.3 云数据管理

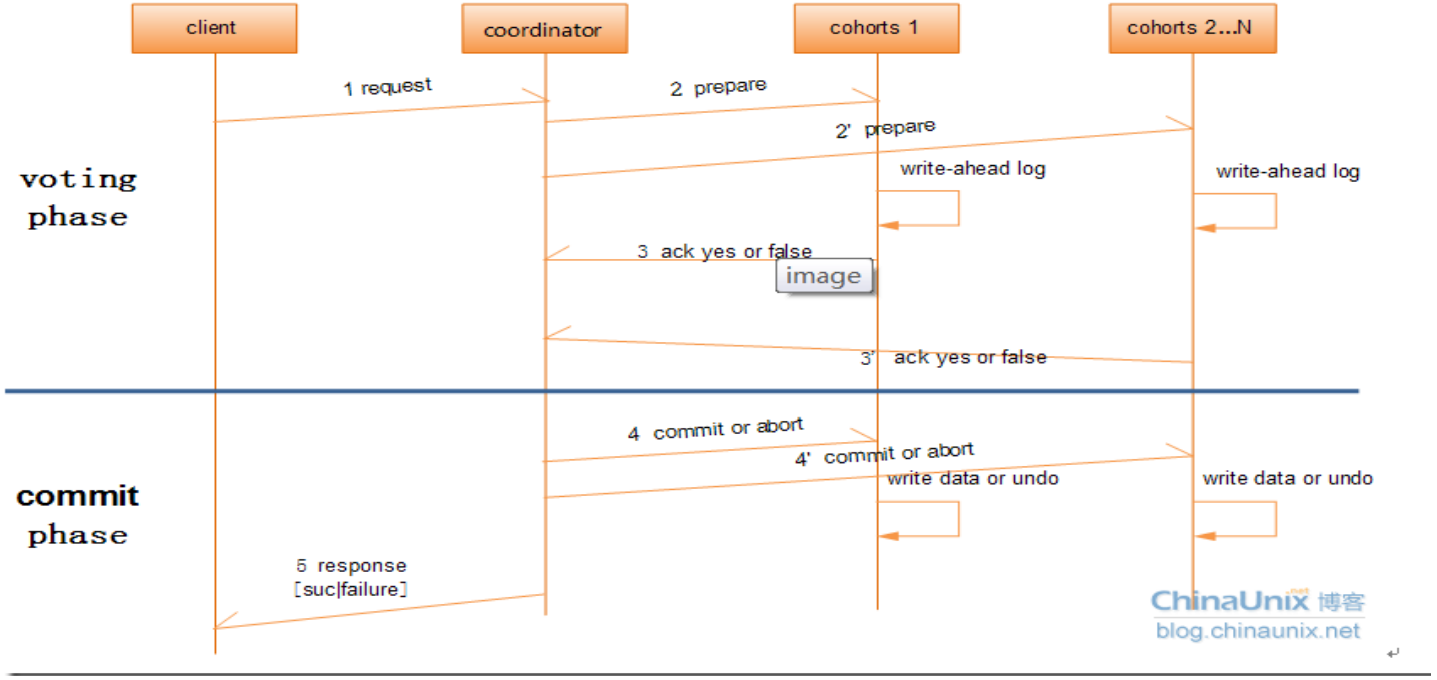
➢一致性模型--最终一致性

- Master-Master(Amazon Dynamo):
  - 一个系统存在两个或多个Master。
  - 每个Master都提供read-write服务。
- 假设某数据三个副本分别存在node1,node2,node3中,(N=3，R=W=2)
- 第一次更新：node1失效，成功更新node2，node3，返回
- 第二次更新：node3失效，成功更新node1，node2，返回
- 这时候node2才是正确数据，node1和node3都只是部分更新
- 此时node2失效，发生读请求，读到node1和node3数据，返回
- 这时候读到的两个版本都只是部分更新，而且还有版本冲突的情况，只能依赖用户自己解决(Vector Clock)。

11.3 云数据管理

➢一致性模型--强一致性

- Two Phase Commit:
  - 在分布式系统中，为了保持事务的ACID特性，需要引入一个作为协调者的组件来统一掌控所有节点(称作参与者)的操作结果并最终指示这些节点是否要把操作结果进行真正的提交(比如将更新后的数据写入磁盘等等)。



11.3 云数据管理

➢一致性模型--强一致性

- Two Phase Commit:
  - 第一阶段：
    - 协调者会问所有的参与者结点，是否可以执行提交操作。
    - 各个参与者开始事务执行的准备工作：如：为资源上锁，预留资源，写undo/redo log……
    - 参与者响应协调者，如果事务的准备工作成功，则回应“可以提交”，否则回应“拒绝提交”。
  - 第二阶段：
    - 如果所有的参与者都回应“可以提交”，那么，协调者向所有的参与者发送“正式提交”的命令。参与者完成正式提交，并释放所有资源，然后回应“完成”，协调者收集各结点的“完成”回应后结束这个Global Transaction。
    - 如果有一个参与者回应“拒绝提交”，那么，协调者向所有的参与者发送“回滚操作”，并释放所有资源，然后回应“回滚完成”，协调者收集各结点的“回滚”回应后，取消这个Global Transaction。

11.3 云数据管理

➢其他数据一致性分类

- Causal consistency(因果一致性)
  - 如果Process A通知Process B它已经更新了数据，那么Process B的后续读取操作则读取A写入的最新值，而与A没有因果关系的C则可以最终一致性。
- Read-your-writes consistency(过程一致性)
  - 如果Process A写入了最新的值，那么Process A的后续操作都会读取到最新值。但是其它用户可能要过一会才可以看到。
- Session consistency(会话一致性)
  - 此种一致性要求客户端和存储系统交互的整个会话阶段保证Read-your-writes consistency
  - 例如：Hibernate的session提供的一致性保证就属于此种。
- Monotonic read consistency(简单读一致性)
  - 此种一致性要求如果Process A已经读取了对象的某个值，那么后续操作将不会读取到更早的值。
- Monotonic write consistency(简单写一致性)
  - 此种一致性保证系统会序列化执行一个Process中的所有写操作。

11.3 云数据管理

➢服务器一致性

- 概念：
  - N：节点个数
  - W：更新数据时需要确认已经被更新的节点个数
  - R：读数据时需要 读取的节点个数
- 如果W+R>N，那么分布式系统就会提供强一致性的保证，因为，读取数据节点和被写入的节点是有重叠的
- 在一个RDBMS的复制模型中(Master/Slave)，假如N=2，W=2，R=1，此时是一种强一致性，但是这样造成的问题就是可用性的减低，因为要想写操作成功，必须要等2个节点都完成以后才可以。

11.3 云数据管理

➢服务器一致性

- 在分布式系统中，一般都要有容错性，因此一般N都大于3，此时根据CAP理论，一致性，可用性和分区容错性最多只能满足两个，因此我们需要在一致性和分区容错性之间做一个平衡。
- 如果要高的一致性，那么就配置为W=N，R=1，这个时候可用性就会大大降低。
- 如果想要高的可用性，那么此时就需要放松一致性的要求，此时可以配置W=1，这样使得写操作延迟最低。

11.3 云数据管理

➢服务器一致性

- 当存储系统保证最终一致性时，存储系统的配置一般是W+R<=N，此时读取和写入操作是不重叠的。
- 不一致窗口就依赖存储系统的异步实现方式，不一致窗口大小也就等于从更新开始到所有节点都异步更新完成之间的时间。
- N,R,W的例子
  - (N,R,W) 的值典型设置为 (3, 2 ,2),兼顾性能与可用性
  - W = 1, R = N,对写操作要求高性能\高可用。
  - R = 1, W = N , 对读操作要求高性能\高可用，比如类似cache之类业务。
  - W = Q, R = Q where Q = N / 2 + 1适用一般应用，读写性能之间取得平衡。如N=3,W=2,R=2

11.3 云数据管理

➢Base性质

- BASE(Basically Available 基本可用,Soft state 软状态,Eventually consistent 最终一致)是对CAP中的C & A的衍生
  - 在单机环境中，数据的属性是ACID
  - 而在分布式环境中，数据的属性就是BASE

- BASE是为了解决关系数据库强一致性引起的问题而引起的可用性降低而提出的解决方案。
- BASE模型不同于ACID模型，它通过牺牲高一致性，获得可用性和可靠性

11.3 云数据管理

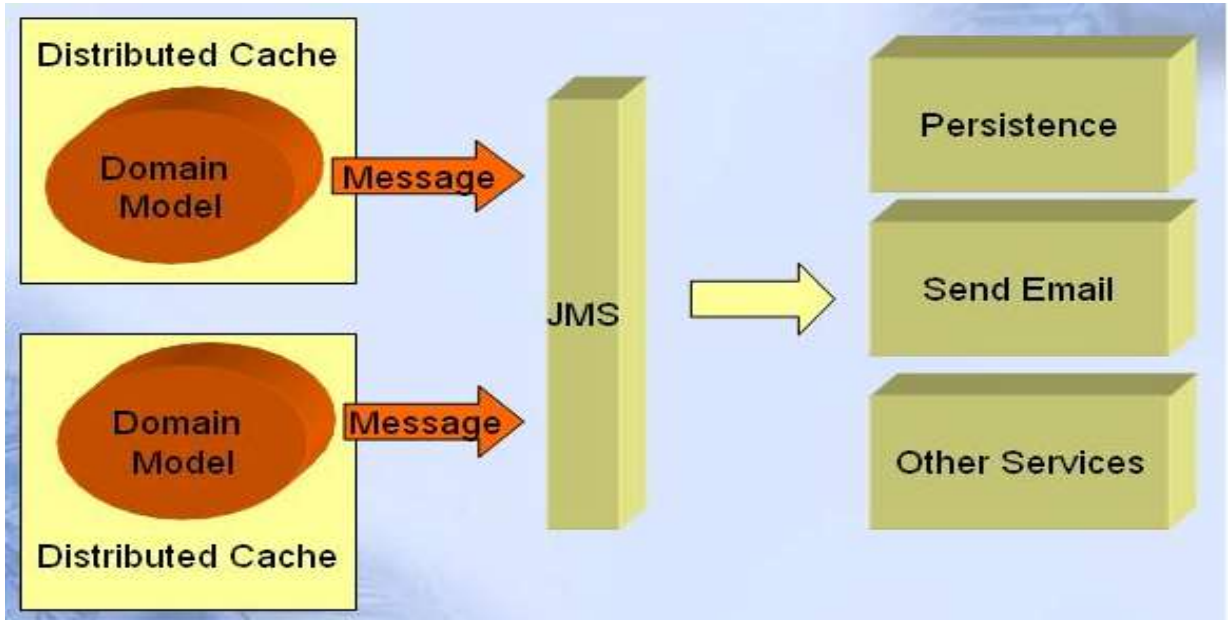
➢Base性质

- 现在NOSQL运动丰富了拓展了BASE思想，可按照具体情况定制特别方案，比如忽视一致性，获得高可用性等等，NOSQL应该有下面两个流派
  - key-value存储，如Amaze Dynamo等，可根据CAP三原则灵活选择不同倾向的数据库产品。
  - 领域模型 + 分布式缓存 + 存储 ，可根据CAP三原则结合自己项目定制灵活的分布式方案，难度高。
- 共同点：都是关系数据库SQL以外的可选方案，逻辑随着数据分布，任何模型都可以自己持久化，将数据处理和数据存储分离，将读和写分离，存储可以是异步或同步，取决于对一致性的要求程度。

11.3 云数据管理

➢Base性质

- 不同点：
  - NOSQL之类的key-value存储产品可以适合非Java如PHP RUBY等领域，是一种可以拿来就用的产品
  - 领域模型 + 分布式缓存 + 存储是一种复杂的架构解决方案，不是产品(如下图)，但这种方式更灵活。



11.3 云数据管理

➢ACID性质

- 关系数据库的ACID模型拥有 高一致性 + 可靠性 丧失可用性(跨数据库事务下):
  - Atomicity原子性.一个事务中所有操作都必须全部完成，要么全部不完成。
  - Consistency一致性. 在事务开始或结束时，数据库应该在一致状态。
  - Isolation隔离性 事务将假定只有它自己在操作数据库，彼此不知晓。
  - Durability. 持久性 一旦事务完成，就不能返回。
  - 跨数据库事务：2PC (two-phase commit)， 2PC is the anti-scalability pattern (Pat Helland) 是反可伸缩模式的，JavaEE中的JTA事务可以支持2PC

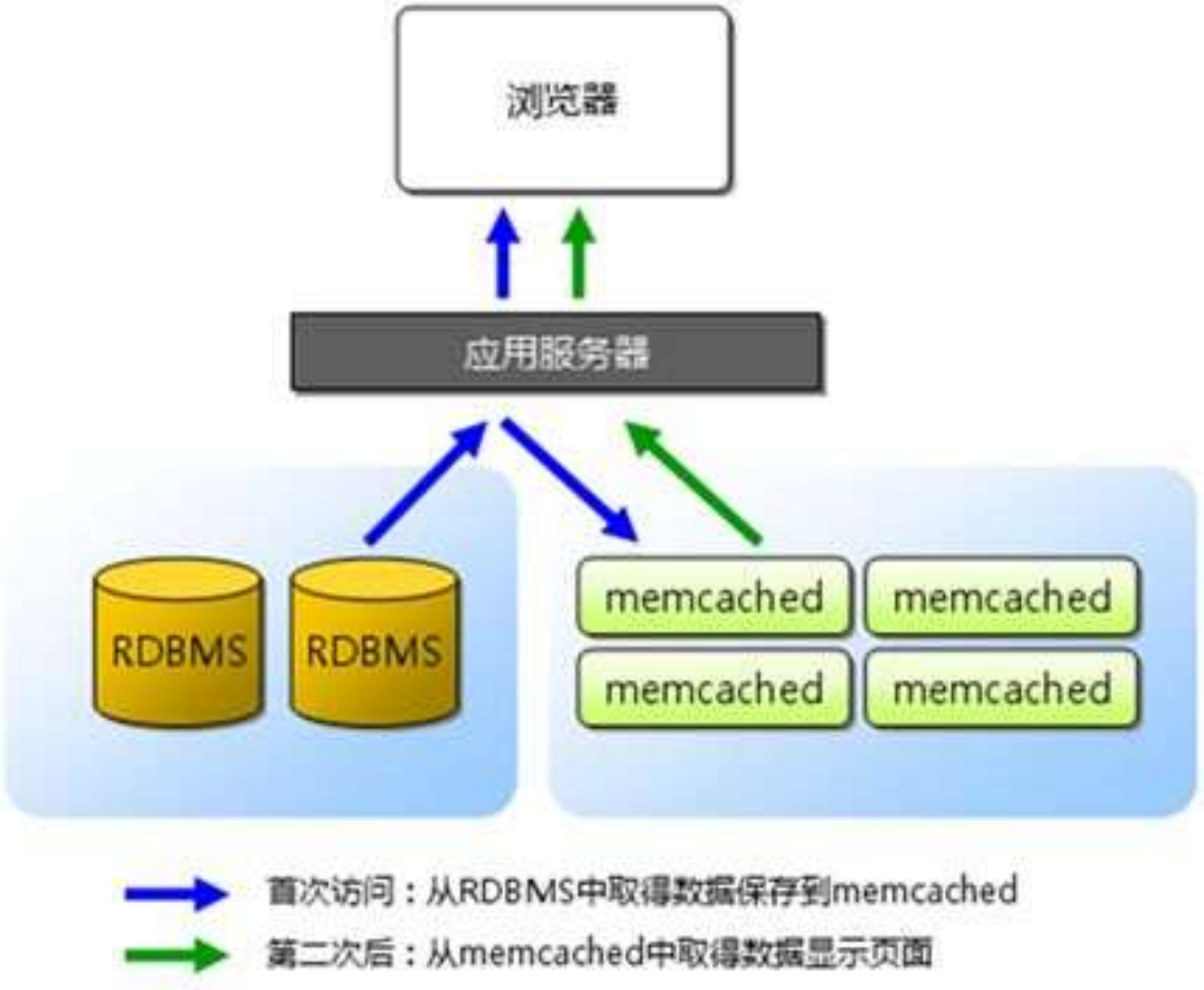
11.3 云数据管理

➢ACID VS BASE



11.3 云数据管理

➢分布式缓存



11.3 云数据管理

➢一致性哈希

- 一致性哈希是一种哈希算法，简单地说在移除或者添加一个服务器时，此算法能够尽可能小地改变已存在的服务请求与处理请求服务器之间的映射关系,，并尽可能满足单调性要求,，从而使得分布式哈希(DHT)可以在P2P环境中得到应用。
- 具体的说，在一致性哈希所构成的存储拓扑中，每个存储节点仅需维护少量相邻节点的信息，并且在新/旧节点加入/退出系统时，仅有相关的少量节点参与到拓扑的维护中， 这使得一致性哈希算法成为一个具有实用意义的DHT(DistributedHash Table，分布式哈希表)算法。

11.3 云数据管理

➢一致性哈希

- 在动态变化的Cache环境中，判定哈希算法好坏的4个性质：
- 平衡性(Balance)：平衡性是指哈希的结果能够尽可能分布到所有的缓冲中去，这样可以使得所有的缓冲空间都得到利用。大多数哈希算法都能够满足这一条件。
- 单调性(Monotonicity)：单调性是指如果已经有一些内容通过哈希分派到了相应的缓冲中，又有新的缓冲加入到系统中，则哈希的结果应能够保证：原有已分配的内容可以被映射到原有的或者新的缓冲中去，而不会被映射到旧的缓冲集合中的其他缓冲区。

11.3 云数据管理

➢一致性哈希

- 单调性(Monotonicity) ——续
  - 简单哈希算法往往不能满足单调性的要求，如最简单的线性哈希：
    - $\mathrm{x} \rightarrow (ax+b) mod(N)$
  - 在上式中，N表示全部缓冲的数目。不难看出，当缓冲大小发生变化时(从N1到N2)，原来所有的哈希结果均会发生变化，从而不满足单调性的要求。
  - 哈希结果的变化意味着当缓冲空间发生变化时，所有的映射关系需要在系统内全部更新。
  - 而在P2P系统内，缓冲的变化等价于Peer加入或退出系统，这一情况在P2P系统中会频繁发生，因此会带来极大计算和传输负荷。单调性就是要求哈希算法能够避免这一情况的发生。

11.3 云数据管理

➢一致性哈希

- 分散性(Spread)
  - 在分布式环境中，终端有可能看不到所有的缓冲，而是只能看到其中的一部分。当终端希望通过哈希过程将内容映射到缓冲上时，由于不同终端所见的缓冲范围有可能不同，从而导致哈希的结果不一致，最终的结果是相同的内容被不同的终端映射到不同的缓冲区 中。这种情况显然是应该避免的，因为它导致相同内容被存储到不同缓冲中去，降低了系统存储的效率。分散性的定义就是上述情况发生的严重程度。好的哈希算法应能够尽量避免不一致的情况发生，也就是尽量降低分散性。
- 负载(Load) 问题
  - 负载问题实际上是从另一个角度看待分散性问题。既然不同的终端可能将相同的内容映射到不同的缓冲区中，那么对于一个特定的缓冲区而言，也可能被不同的用户映射为不同的内容。与分散性一样，这种情况也是应当避免的，因此好的哈希算法应能够尽量降低缓冲的负荷。

11.3 云数据管理

➢一致性哈希

- 从表面上看，一致性哈希针对的是分布式缓冲的问题，但是如果将缓冲看作P2P系统中的Peer，将映射的内容看作各种共享的资源(数据，文件，媒体流等)，就会发现两者实际上是在描述同一问题。

11.3 云数据管理

➢一致性哈希:例

- 例：假定有一个分布式WEB缓存系统，那么其数据缓存的算法可以有两种。
- hash模余算法：
  - 根据hash(key)% N的结果决定存储到哪个节点(key:数据的关键字键值，N:服务器个数)，此计算方法简单，数据的分散性也相当优秀。
  - 其缺点是当添加或移除服务器时，缓存重组的代价巨大。添加/删除服务器后(或者是某台服务器出现故障之后)，余数就会产生巨变，这样就无法保证获取时计算的服务器节点与保存时相同，从而影响缓存的命中率——造成原有的缓存数据大规模失效。

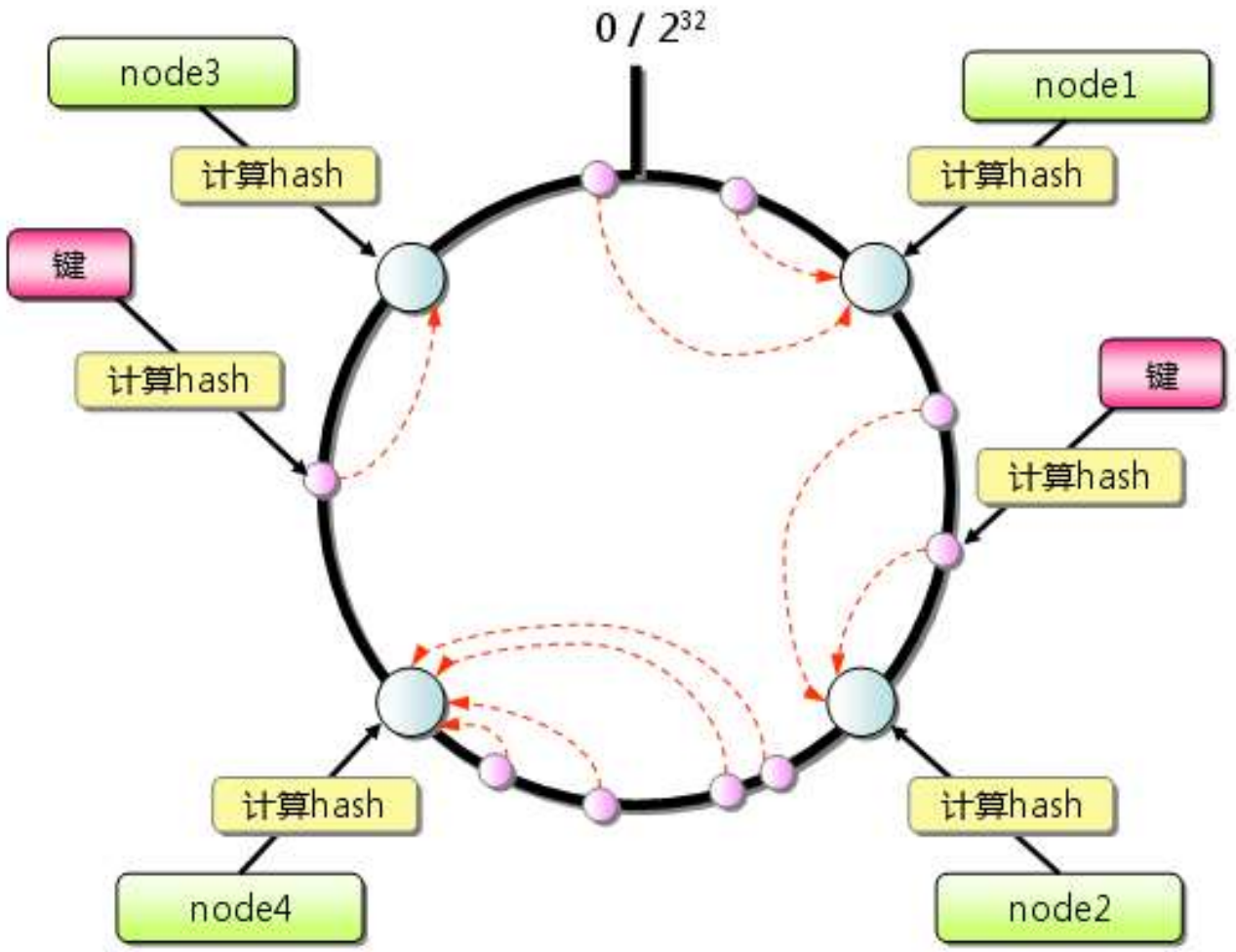
11.3 云数据管理

➢一致性哈希:例

- 一致性哈希方法
  - 我们采用了一种新的方式来解决问题，处理服务器的选择不再仅仅依赖key的hash本身而是将服务实例(节点)的配置也进行hash运算。
  - 首先，求出每个服务节点的hash，并将其配置到一个$0 \sim 2^{32}-1$的圆环区间(continuum)上。
  - 其次，使用同样的方法求出所需要存储的key的hash，也将其配置到这个圆环区间(continuum)上。
  - 然后，从数据映射到的位置开始顺时针查找，将数据保存到找到的第一个服务节点上。如果超过$2^{32}-1$仍然找不到服务节点，就会保存到第一个服务节点上。

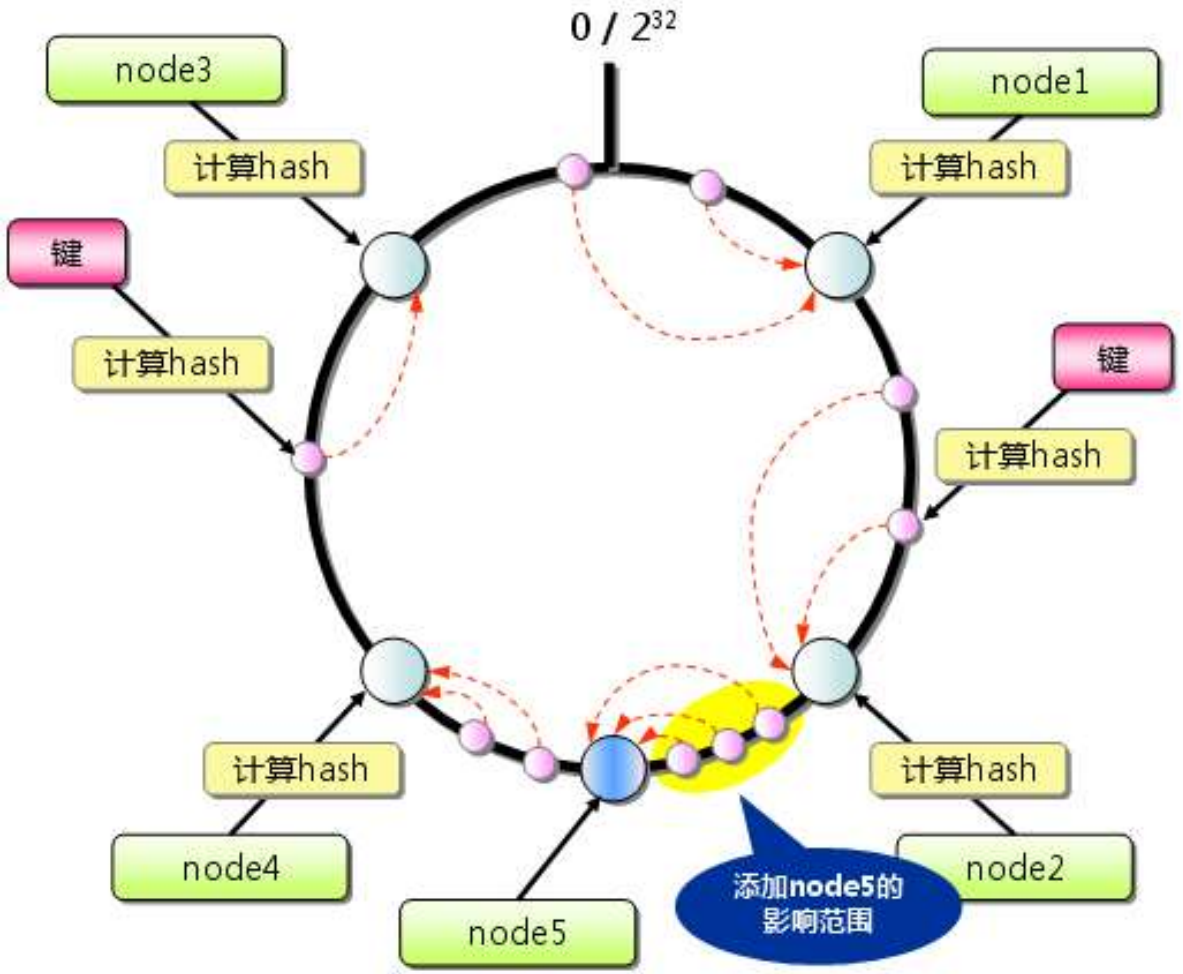
11.3 云数据管理

➢一致性哈希:示意图



11.3 云数据管理

➢一致性哈希:示意图



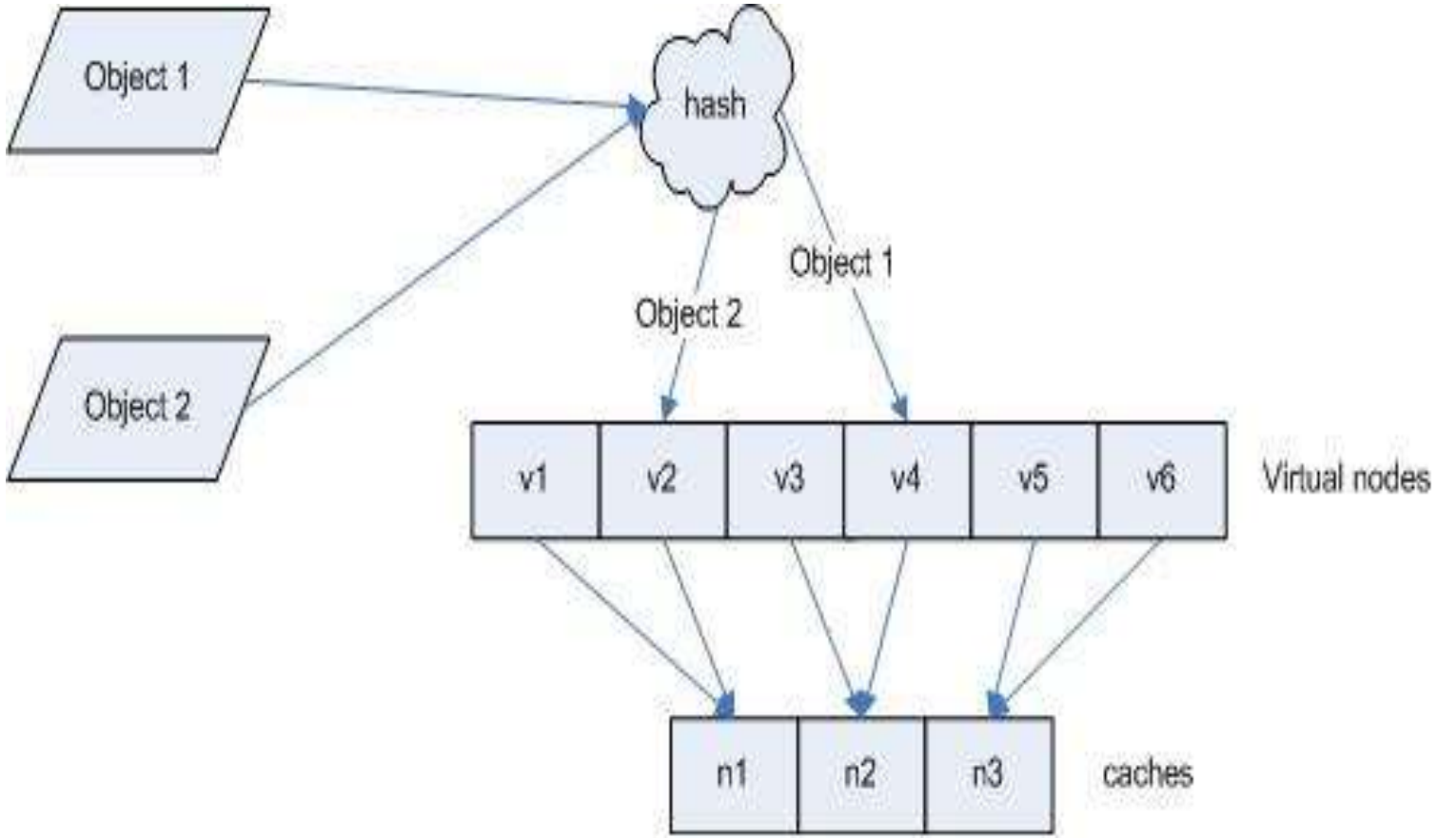
11.3 云数据管理

➢一致性哈希的改进:虚拟节点

- 一致性哈希具有随机性，当节点数量较少时节点在环上分布不够均匀，对于节点可能存在的性能差异也没有考虑。
- 为解决这个问题，以 Dynamo 为代表，提出了基于虚拟节点的改进算法，其核心思路是引入虚拟节点，每个虚拟节点都有一个对应的物理节点，而每个物理节点可以对应若干个虚拟节点。

11.3 云数据管理

➢虚拟节点的hash



11.3 云数据管理

➢一致性哈希的改进:虚拟节点

- Dynamo 将圆环划分成了M 等分，若加入物理节点为 N 个(N<M)，则每个物理节点拥有 V=M/N 个节点数。
- 当有物理节点离线时，由于该节点对应的虚拟节点均匀地分布在环上，其附近的节点将会均匀地分担这个节点的原有负载，当有新节点加入时，同理，其他节点的负担也将均匀转移到其上。

11.3 云数据管理

➢一致性哈希的改进:其他

- 此外，依据物理节点的实际性能为权值分配环上的虚拟节点数目给物理节点，可以解决存储节点性能差异的问题。
- 为解决数据迁移带来的效率降低问题，可以将新增节点完成的数据迁移分布在每一次的查询任务中去，相当于每一次查询都可以迁移一小部分数据，此外，还可以在系统闲暇时间进行数据迁移，这样可以有效地提高效率。
- 优点：发生单点故障时负载会均衡分散到其他所有节点，程序实现也比较优雅。

11.3 云数据管理

➢一致性哈希的应用:路由算法

- 为了构建查询所需的路由，一致性哈希要求每个节点存储其上行节点(ID值大于自身的节点中最小的)和下行节点(ID值小于自身的节点中最大的)的位置信息 (IP地址)。
- 当节点需要查找内容时，就可以根据内容的键值决定向上行或下行节点发起查询请求。
- 收到查询请求的节点如果发现自己拥有被请求的目标，可以直接向发起查询请求的节点返回确认；
- 如果发现不属于自身的范围，可以转发请求到自己的上行/下行节点。

11.3 云数据管理

➢一致性哈希的应用:路由算法

- 为了维护上述路由信息，在节点加入/退出系统时，相邻的节点必须及时更新路由信息。
- 这就要求节点不仅存储直接相连的下行节点位置信息，还要知道一定深度 (n跳) 的间接下行节点信息，并且动态地维护节点列表。
- 当节点退出系统时，它的上行节点将尝试直接连接到最近的下行节点，连接成功后，从新的下行节点获得下行节点列表并更新自身的节点列表。
- 同样的，当新的节点加入到系统中时，首先根据自身的ID找到下行节点并获得下行节点列表，然后要求上行节点修改其下行节点列表，这样就恢复了路由关系。

11.3 云数据管理

➢一致性哈希的应用:分布式哈希表(DHT)

- 分布式哈希表技术(Distributed Hash Table,DHT)是一种分布式存储技术。
- 在不需要服务器的情况下，每个客户端负责一个小范围的路由，并负责存储一小部分数据，从而实现整个DHT网络的寻址和存储。

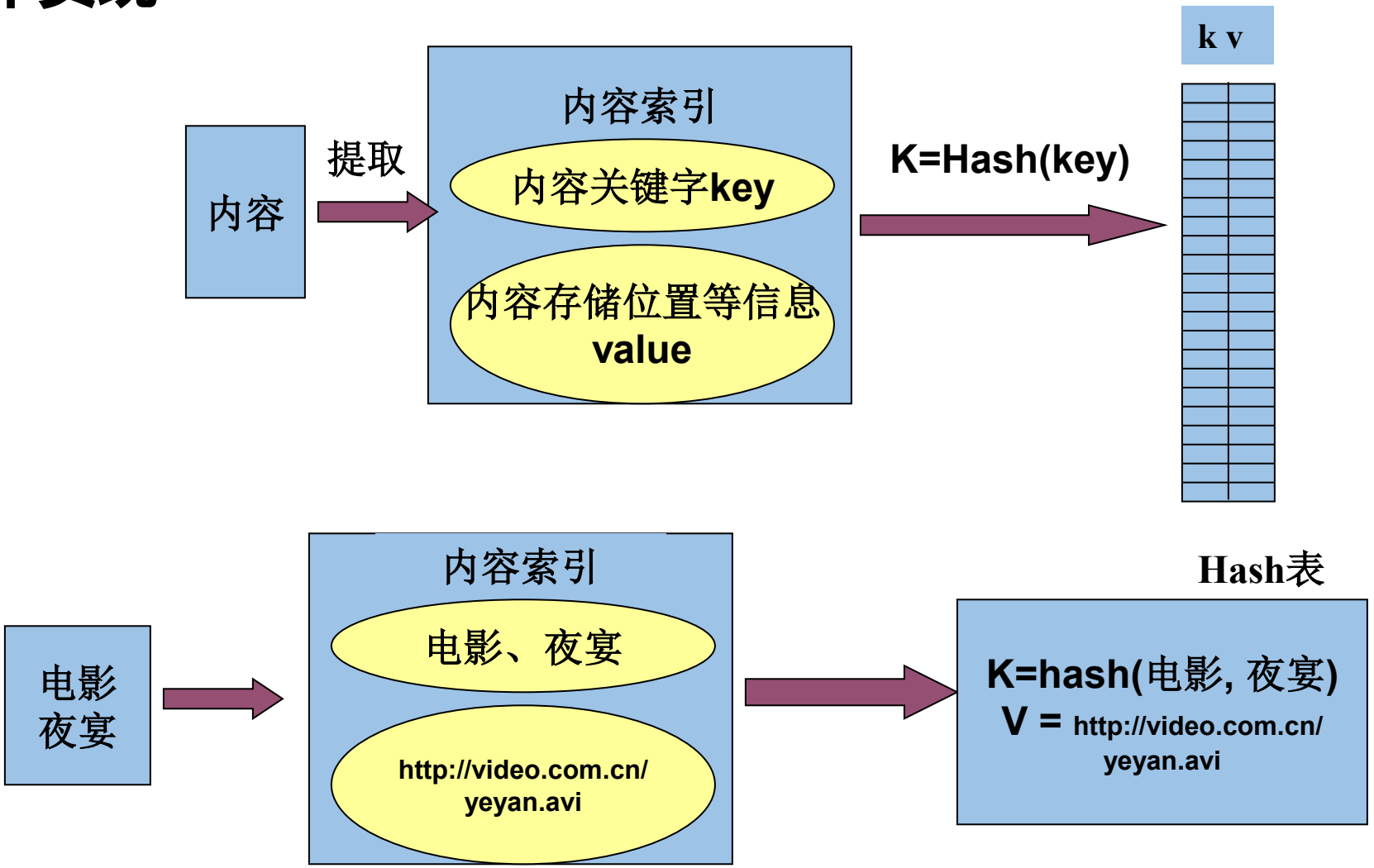
11.3 云数据管理

➢DHT具体步骤

- 将内容索引抽象为<K, V>对:
  - K是内容关键字的Hash摘要
  - K = Hash(key)
  - V是存放内容的实际位置，例如节点IP地址等
- 所有的<K, V>对组成一张大的Hash表，因此该表存储了所有内容的信息
- 每个节点都随机生成一个标识(ID)，把Hash表分割成许多小块，按特定规则(即K和节点ID之间的映射关系)分布到网络中，节点按这个规则在应用层上形成一个结构化的重叠网络(Overlay)
- 给定查询内容的K值，可以根据K和节点ID之间的映射关系在重叠网络上找到相应的V值，从而获得存储文件的节点IP地址

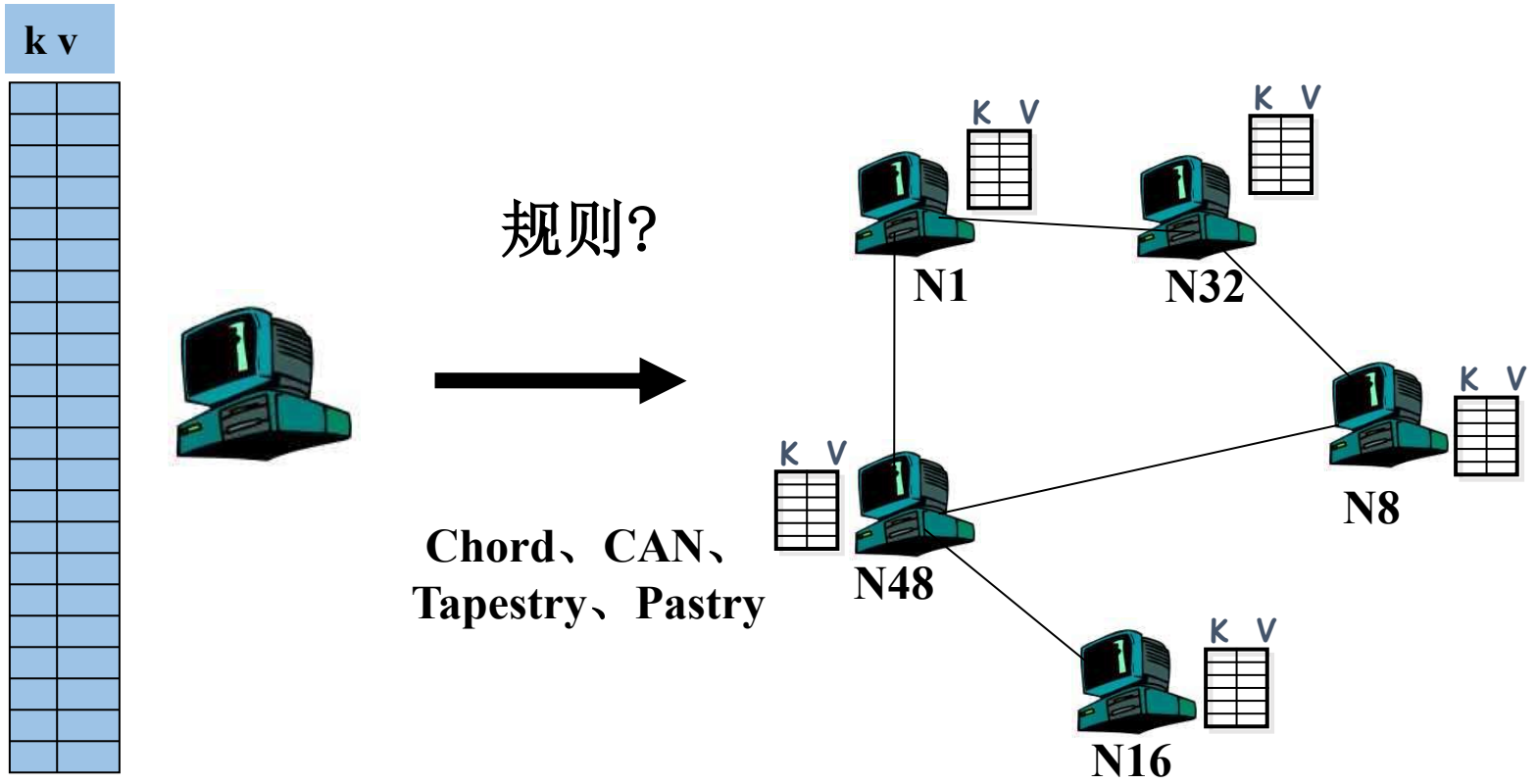
11.3 云数据管理

➢DHT具体实现



11.3 云数据管理

➢DHT具体实现



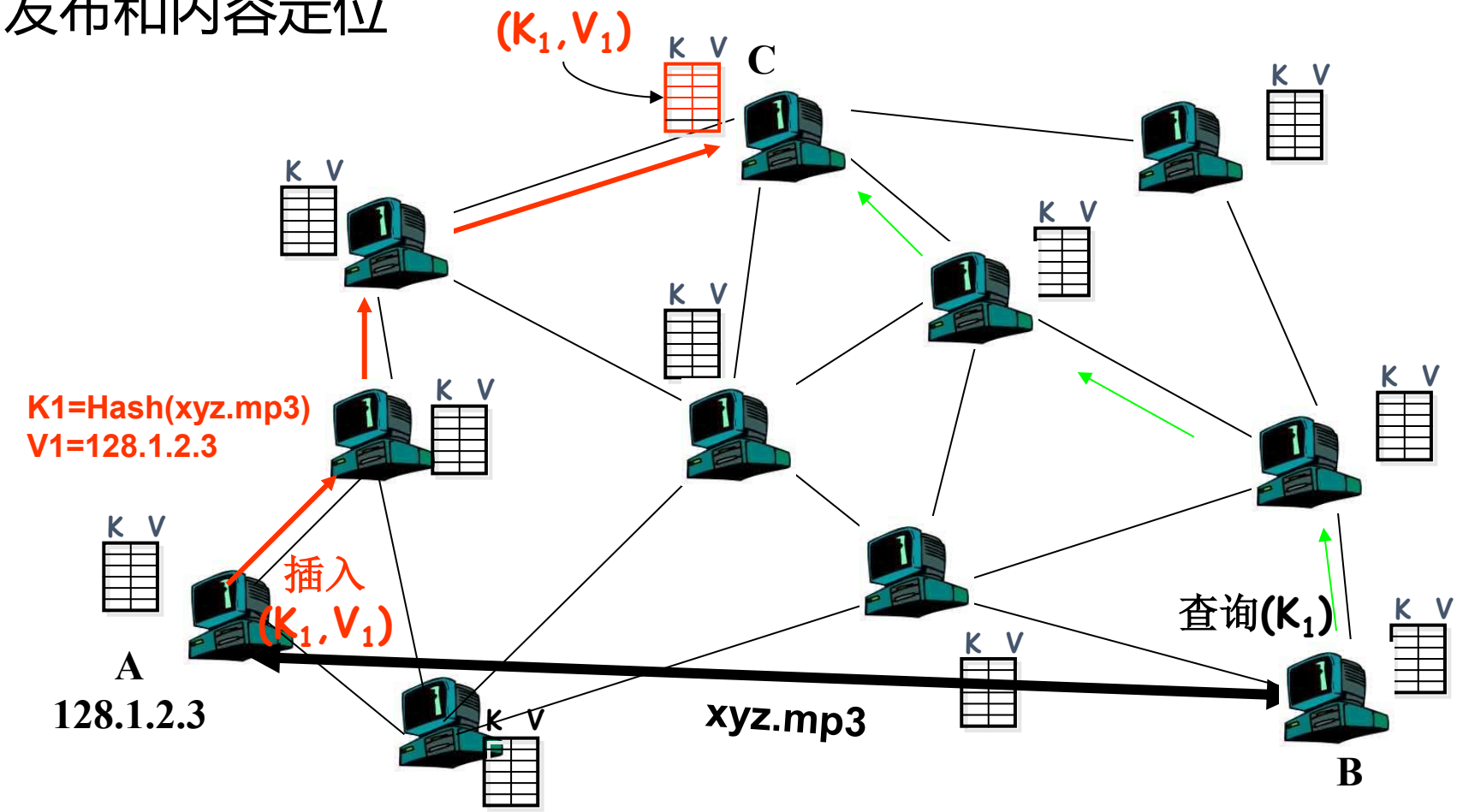
a. Hash表　　b. 分布式Hash表

在许多情况下,节点ID为节点IP地址的Hash摘要

11.3 云数据管理

➢DHT 具体实现

- 索引发布和内容定位



11.3 云数据管理

➢DHT具体实现

- 定位(Locating)
  - 节点ID和其存放的<K, V>对中的K存在着映射关系，因此可以由K获得存放该<K, V>对的节点ID。
- 路由(Routing)
  - 在重叠网上根据节点ID进行路由，将查询消息最终发送到目的节点。每个节点需要有到其邻近节点的路由信息，包括节点ID、IP等。
- 网络拓扑
  - 拓扑结构由节点ID和其存放的<K, V>对中的K之间的映射关系决定。
  - 拓扑动态变化，需要处理节点加入/退出/失效的情况。

在重叠网上节点始终由节点ID标识，并且根据ID进行路由

11.4 NoSQL数据库

➢NoSQL概述



概念演变 →

Not only SQL

最初表示“反SQL”运动
用新型的非关系数据库取代关系数据库

现在表示关系和非关系型数据库各有优缺点
彼此都无法互相取代

- 通常，NoSQL数据库具有以下几个特点：
  - 灵活的可扩展性
  - 灵活的数据模型
  - 与云计算紧密融合

11.4 NoSQL数据库

➢NoSQL兴起的原因

- 关系数据库已经无法满足Web2.0的需求。主要表现在以下几个方面：
  - 无法满足海量数据的管理需求
  - 无法满足数据高并发的需求
  - 无法满足高可扩展性和高可用性的需求

11.4 NoSQL数据库

➢NoSQL与关系数据库的比较

表5-1 NoSQL和关系数据库的简单比较

比较标准	RDBMS	NoSQL	备注
数据库原理	完全支持	部分支持	RDBMS有关系代数理论作为基础 NoSQL没有统一的理论基础
数据规模	大	超大	RDBMS很难实现横向扩展，纵向扩展的空间也比较有限，性能会随着数据规模的增大而降低 NoSQL可以很容易通过添加更多设备来支持更大规模的数据
数据库模式	固定	灵活	RDBMS需要定义数据库模式，严格遵守数据定义和相关约束条件 NoSQL不存在数据库模式，可以自由灵活定义并存储各种不同类型的数据
查询效率	快	可以实现高效的简单查询，但是不具备高度结构化查询等特性，复杂查询的性能不尽人意	RDBMS借助于索引机制可以实现快速查询(包括记录查询和范围查询) NoSQL没有索引，虽然NoSQL可以使用MapReduce来加速查询，但是，在复杂查询方面的性能仍然不如RDBMS

11.4 NoSQL数据库

➢NoSQL与关系数据库的比较

表5-1 NoSQL和关系数据库的简单比较(续)

比较标准	RDBMS	NoSQL	备注
一致性	强一致性	弱一致性	RDBMS严格遵守事务ACID模型，可以保证事务强一致性 NoSQL放松对事务ACID四性的要求，而是遵守BASE模型，只能保证最终一致性
数据完整性	容易实现	很难实现	任何一个RDBMS都可以很容易实现数据完整性，比如通过主键或者非空约束来实现实体完整性，通过主键、外键来实现参照完整性，通过约束或者触发器来实现用户自定义完整性 但是，在NoSQL数据库却无法实现
扩展性	一般	好	RDBMS很难实现横向扩展，纵向扩展的空间也比较有限 NoSQL在设计之初就充分考虑了横向扩展的需求，可以很容易通过添加廉价设备实现扩展
可用性	好	很好	RDBMS在任何时候都以保证数据一致性为优先目标，其次才是优化系统性能，随着数据规模的增大，RDBMS为了保证严格的一致性，只能提供相对较弱的可用性 NoSQL任何时候都能提供较高的可用性

11.4 NoSQL数据库

➢NoSQL与关系数据库的比较

表5-1 NoSQL和关系数据库的简单比较(续)

比较标准	RDBMS	NoSQL	备注
标准化	是	否	RDBMS已经标准化(SQL) NoSQL还没有行业标准，不同的NoSQL数据库都有自己的查询语言，很难规范应用程序接口
技术支持	高	低	RDBMS经过几十年的发展，已经非常成熟，Oracle等大型厂商都可以提供很好的技术支持 NoSQL在技术支持方面仍然处于起步阶段，还不成熟，缺乏有力的技术支持
可维护性	复杂	复杂	RDBMS需要专门的数据库管理员(DBA)维护 NoSQL数据库虽然没有DBMS复杂，也难以维护

11.4 NoSQL数据库

➢NoSQL与关系数据库的比较

- 总结：
  - 关系数据库和NoSQL数据库各有优缺点，彼此无法取代
    - 关系数据库应用场景：电信、银行等领域的关键业务系统，需要保证强事务一致性
    - NoSQL数据库应用场景：互联网企业、传统企业的非关键业务(比如数据分析)
  - 采用混合结构
    - 案例：亚马逊公司就使用不同类型的数据库来支撑它的电子商务应用
    - 对于“购物篮”这种临时性数据，采用键值存储会更加高效
    - 当前的产品和订单信息则适合存放在关系数据库中。
    - 大量的历史订单信息则适合保存在类似MongoDB的文档数据库中

11.4 NoSQL数据库

➢NoSQL的类型

类型	部分代表	特点
列存储	Hbase Cassandra Hypertable	顾名思义，是按列存储数据的。最大的特点是方便存储结构化和半结构化数据，方便做数据压缩，对针对某一列或者某几列的查询有非常大的IO优势。
文档存储	MongoDB CouchDB	文档存储一般用类似json的格式存储，存储的内容是文档型的。这样也就有有机会对某些字段建立索引，实现关系数据库的某些功能。
key-value存储	Tokyo Cabinet / Tyrant Berkeley DB MemcacheDB Redis	可以通过key快速查询到其value。一般来说，存储不管value的格式，照单全收。(Redis包含了其他功能)
图存储	Neo4J FlockDB InfoGrid	图形关系的最佳存储。使用传统关系数据库来解决的话性能低下，而且设计使用不方便。
对象存储	db4o Versant	通过类似面向对象语言的语法操作数据库，通过对象的方式存取数据。
xml数据库	Berkeley DB XML BaseX	高效的存储XML数据，并支持XML的内部查询语法，比如XQuery,Xpath。

11.4 NoSQL数据库

➢键值存储

相关产品	Redis、Riak、SimpleDB、Chordless、Scalaris、Memcached
数据模型	键/值对
典型应用	内容缓存，比如会话、配置文件、参数、购物车等
优点	扩展性好，灵活性好，大量写操作时性能高
缺点	无法存储结构化信息，条件查询效率较低
使用者	百度云数据库(Redis)、GitHub(Riak)、BestBuy(Riak)、Twitter(Redis和Memcached)、StackOverFlow(Redis)、Instagram (Redis)、Youtube(Memcached)、Wikipedia(Memcached)

11.4 NoSQL数据库

➢列族数据库

相关产品	BigTable、HBase、Cassandra、HadoopDB、GreenPlum、PNUTS
数据模型	列族
典型应用	分布式数据存储与管理
优点	查找速度快，可扩展性强，容易进行分布式扩展，复杂性低
缺点	功能较少，大都不支持强事务一致性
使用者	Ebay(Cassandra)、Instagram(Cassandra)、NASA(Cassandra)、Twitter(Cassandra and HBase)、Facebook(HBase)、Yahoo!(HBase)

11.4 NoSQL数据库

➢文档数据库

相关产品	CouchDB、MongoDB、Terrastore、ThruDB、RavenDB、SisoDB、RaptorDB、CloudKit、Perservere、Jackrabbit
数据模型	版本化的文档
典型应用	存储、索引并管理面向文档的数据或者类似的半结构化数据
优点	性能好，灵活性高，复杂性低，数据结构灵活
缺点	缺乏统一的查询语法
使用者	百度云数据库(MongoDB)、SAP (MongoDB)、Codecademy (MongoDB)、Foursquare (MongoDB)、NBC News (RavenDB)

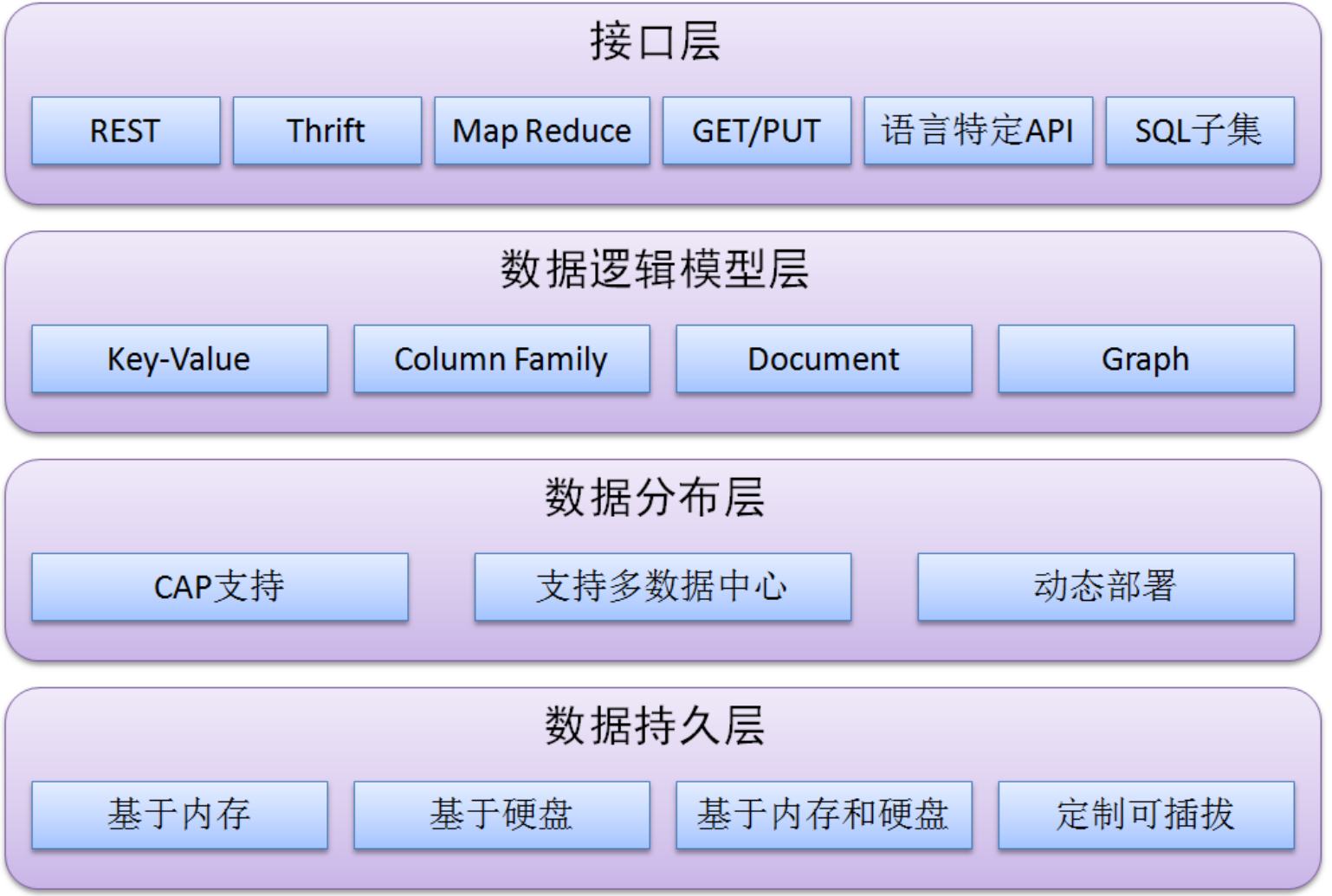
11.4 NoSQL数据库

➢图数据库

相关产品	Neo4J、OrientDB、InfoGrid、Infinite Graph、GraphDB
数据模型	图结构
典型应用	应用于大量复杂、互连接、低结构化的图结构场合，比如社交网络、推荐系统等
优点	灵活性高，支持复杂的图形算法，可用于构建复杂的关系图谱
缺点	复杂性高，只能支持一定的数据规模
使用者	Adobe(Neo4J)、Cisco(Neo4J)、T-Mobile(Neo4J)

11.4 NoSQL数据库

➢NoSQL的整体架构



11.4 NoSQL数据库

➢接口层(Interfaces)

- 这层的主要作用是为上层应用提供合适和方便的数据调用接口，而且提供的选择远多于传统的关系型数据库，主要有六大类接口：
- 其一是常见的 REST(Representational State Transfer)，采用REST的产品有HBase和CouchDB等。
- 其二是源自Facebook的RPC协议Thrift，支持Thrift的产品 有HBase和Cassandra等。
- 其三是用于大规模数据处理的Map Reduce，其相关产品有HBase，CouchDB和MongoDB等。
- 其四是类似于Memcached的Get/Put方式，采用Get/Put的 产品有Voldemort等。
- 其五是提供语言特定(Language Specific)的API，比如Java，在这方面做的不错是MongoDB。
- 最后一个是提供SQL的子集，虽然“Join”在NoSQL属于禁忌，但 是提供一个SQL的基本子集来方便用户也是一个不错的想法。

11.4 NoSQL数据库

➢逻辑数据模型层(Logical Data Model)

- 这层的主要作用是描述数据的逻辑表现形式，而且与关系型数据库相比NoSQL在逻辑表现形式方面相当灵活，主要有四种形式：
- 最普通的Key- Value形式，这种形式在表现形式是比较单一，但是在扩展方面很有优势，采用Key-Value形式产品的有Voldemort等。
- 列式 (Column Family)，这种形式与Key-Value相比其能支持更复杂的数据，但是在扩展方面稍逊一筹，其相关产品有BigTable，HBase和 Cassnadra等。
- 文档(Doucument)形式，文档形式源自于著名协作软件Lotus Notes，并且本质上与Key-Value形式非常相似，主要区别在于是那个Value只能存储文档形式的数据，同时文档形式在对复杂数据的支持和扩展 这两方面表现都还可以，采用文档形式的产品有Couch DB和MongoDB等。
- 图(Graph)式，图式的使用场景不是很广，主要是为基于图数据结构的数据“度身定做”的，比如SNS应用中的关系等，其最知名的产品就是 Neo4j。

11.4 NoSQL数据库

➢数据分布层(Data Distribution Model)

- 这层的主要作用是定义了数据是如何分布的，和关系型数据库不同的是NoSQL数据库可选择的机制比较多，主要有三种：
- 其一是用于水平扩展的CAP机制，支 持CAP的产品有HBase，MongoDB和Cassandra等。
- 其二是对多数据中心的支持，通过这个机制能够保证横跨多数据中心的NoSQL数据库 能非常平稳地运行，相关的产品有Cassandra等。
- 其三是支持动态部署，也就是能在一个生产集群中能动态并且平滑地添加或者删去一个节点

11.4 NoSQL数据库

➢数据持久层(Data Persistence)

- 这层主要作用是定义了数据的存储形式，主要有四种形式：
- 基于内存形式，这种形式速度最快，但是存在丢失数据的可能性，采 用内存的有Redis等。
- 基于硬盘形式，这种形式，在数据耐久性方面表现不错，但是在速度方面远不如基于内存，其相关产品有MongoDB等。
- 基于内存和硬盘混合形式，因为这种形式主要结合前面两者的优点，所以其在速度上表现不错，同时数据也不会丢失，而且常被认为是最合适的方案，采用这种形式的产品 有HBase和Cassandra等。
- 定制可插拔(Custom Pluggable)形式，这种形式以灵活著称。

11.4 NoSQL数据库

➢从NoSQL到NewSQL数据库

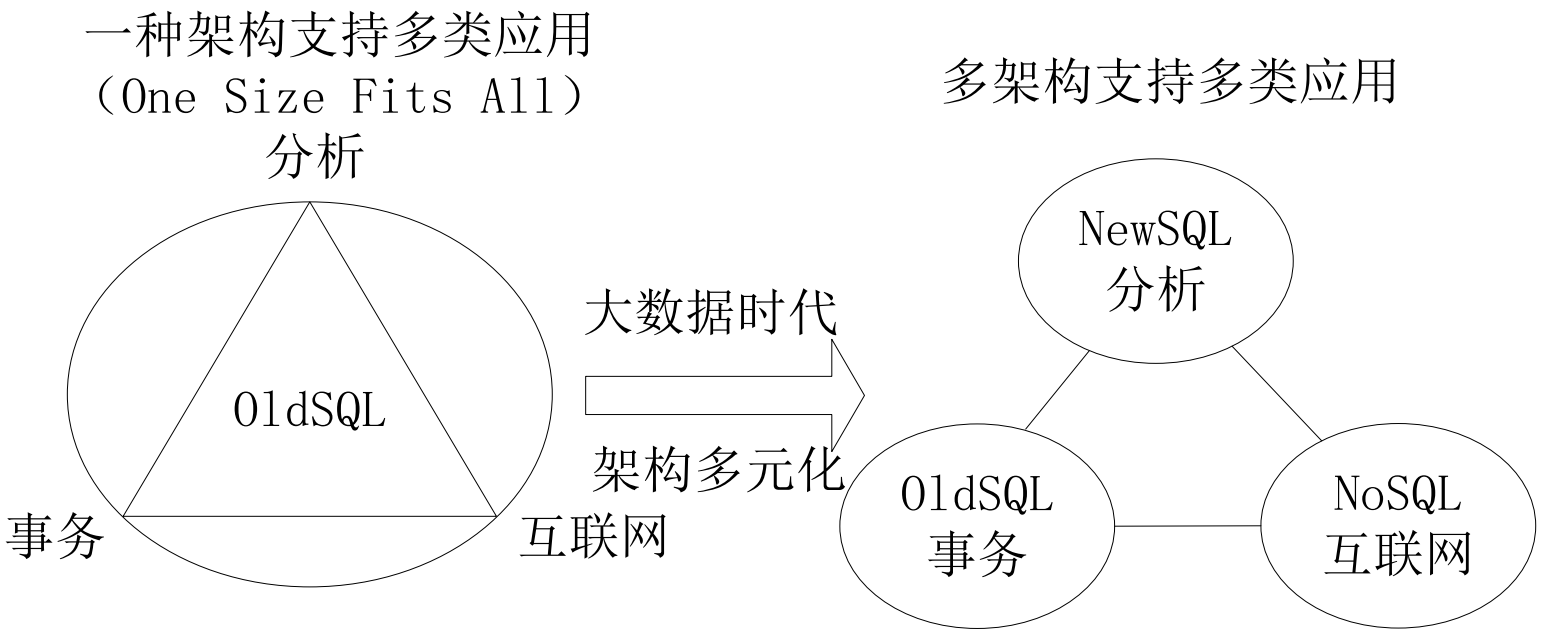


图11-6 大数据引发数据处理架构变革

11.4 NoSQL数据库

➢从NoSQL到NewSQL数据库

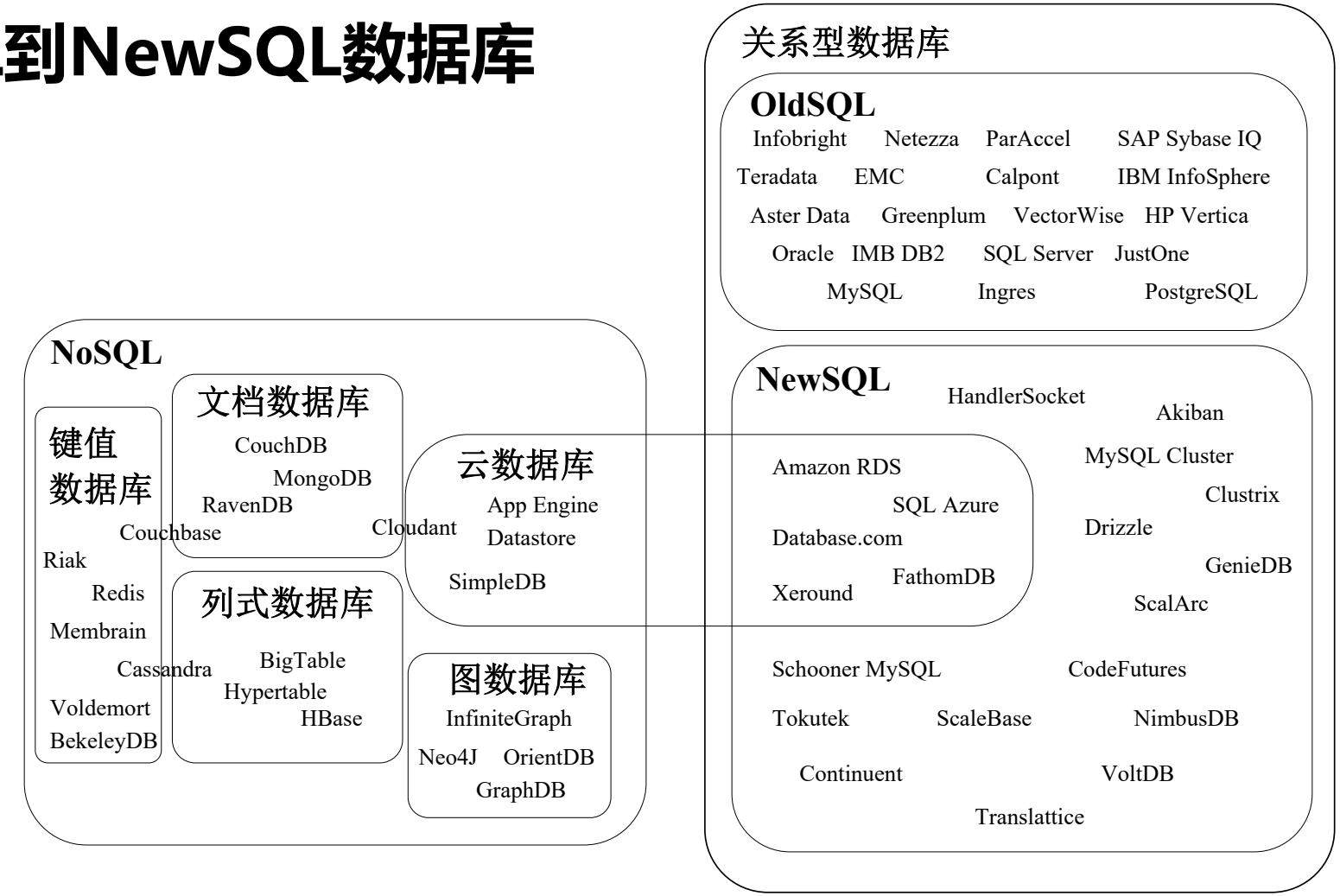


图11-7 关系数据库、NoSQL和NewSQL数据库产品分类图

11.4 NoSQL数据库

➢主要的NoSQL数据库

- BigTable/HBase：在数据模型上面属于Column-Family，采用了Single Master的分布式架构，主要为了存储海量的数据，不强调低延迟。
- Cassandra：在数据模型方面继承BigTable，也是Column-Family，采用Dynamo的机制，其分布式架构类似P2P。
- Redis：是Key-Value的，对List和Set这些操作有原生的支持，由于数据集都是放置于内存中，所以读写速度非常快，但是对分布式支持非常有限。
- MongoDB：是Document DB，提供功能相对而言，比较完善，在分布式方面，有Replica Sets这样的新一代Master/Slave Replication机制。

11.4 NoSQL数据库

➢BigTable

- 简介
  - Bigtable是一个分布式的结构化数据存储系统，它被设计用来处理海量数据：通常是分布在数千台普通服务器上的PB级的数据。Google的很多项目使用Bigtable存储数据，包括Web索引、Google Earth等
- 应用场景
  - 建立互联网索引
    - 爬虫持续不断地抓取新页面，这些页面每页一行地存储到BigTable里
    - MapReduce计算作业运行在整张表上，生成索引，为网络搜索应用做准备
  - 搜索互联网
    - 用户发起网络搜索请求
    - 网络搜索应用查询建立好的索引，从BigTable得到网页
    - 搜索结果提交给用户

11.4 NoSQL数据库

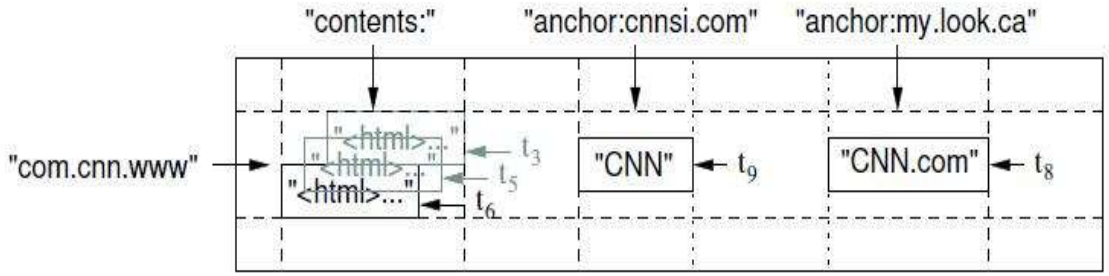
➢BigTable

- 数据模型
  - Bigtable是一个稀疏的、分布式的、持久化存储的多维度排序Map。
  - Map的索引是行关键字、列关键字以及时间戳；Map中的每个value都是一个未经解析的byte数组
- (row:string, column:string,time:int64)->string

11.4 NoSQL数据库

➢BigTable

- 数据模型
  - 一个存储Web网页的例子的表的片断
  - 行名：” com.cnn.www”
  - contents列族：存放的是网页的内容
  - anchor列族：存放引用该网页的锚链接文本。
  - “anchor:cnnsi.com”列表示被cnnsi.com引用
  - “anchhor:my.look.ca”列表示被my.look.ca引用
  - (” com.cnn.www”,” anchor: my.look.ca”,t8)->” CNN.com”



11.4 NoSQL数据库

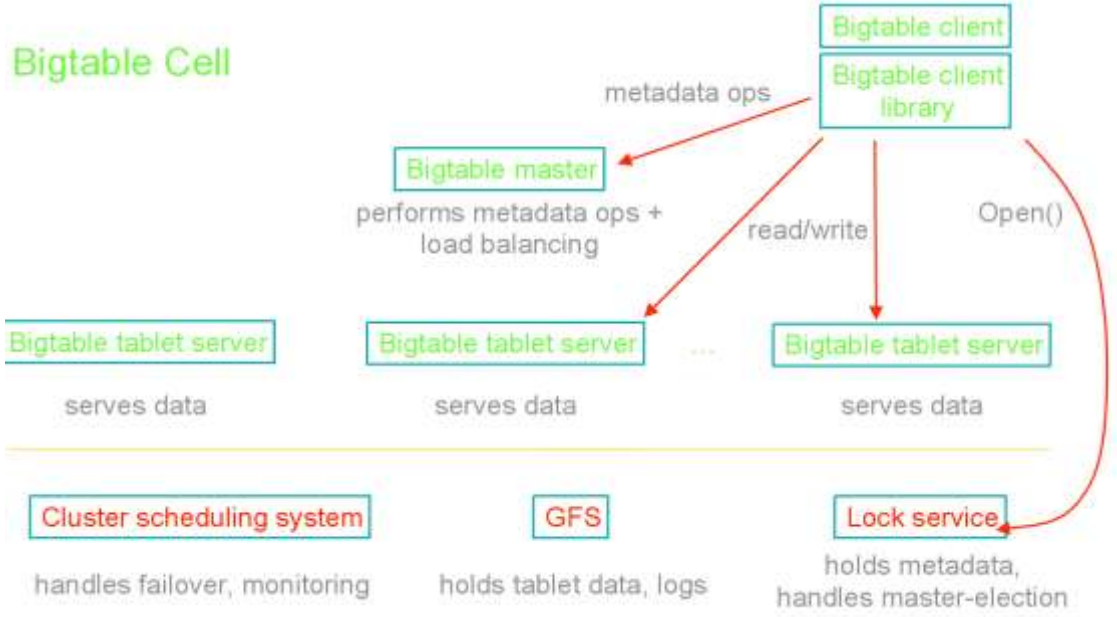
➢BigTable

- 技术要点
  - 基础：GFS, Chubby, SSTable。
  - BigTable使用Google的分布式文件系统(GFS)存储日志文件和数据文件
  - Chubby是一个高可用的、序列化的分布式锁服务组件
  - BigTable内部存储数据的文件是Google SSTable格式的。
  - 元数据组织：chubby->metadata->tablet。
  - 元数据与数据都保存在 Google FS 中，客户端通过 Chubby 服务获得表格元数据的位置。

11.4 NoSQL数据库

➢BigTable

- 技术要点
  - 数据维护与访问：master server 将每个 tablet 的管理责任分配给各个 tablet server，tablet 的分布信息都保存在元数据中，所以客户端无须通过 master 来访问数据，只需要直接跟 tablet server 通信。



11.4 NoSQL数据库

➢BigTable

- 技术要点
  - Log-structured 数据组织：写操作不直接修改原有的数据，而只是将一条记录添加到 commit log 的末尾，读操作需要从 log 中 merge 出当前的数据版本。
  - 具体实现：SSTable, Memtable。

11.4 NoSQL数据库

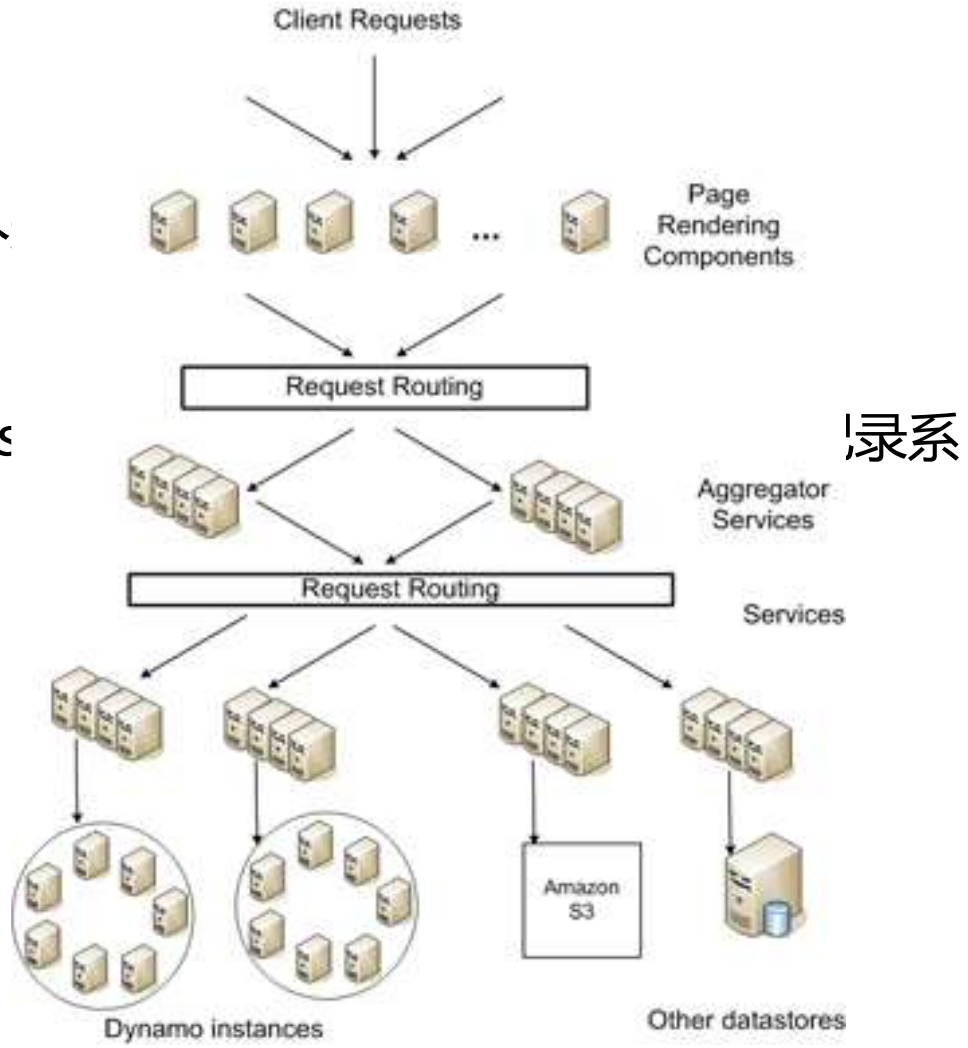
➢BigTable

- 特点：
  - 适合大规模海量数据，PB级数据；
  - 分布式、并发数据处理，效率极高；
  - 易于扩展，支持动态伸缩；
  - 适用于廉价设备；
  - 适合于读操作，不适合写操作。
  - 不适用于传统关系数据库；

11.4 NoSQL数据库

➢Dynamo

- 简介
  - Dynamo 最初是 Amazon 所使用的一个私有的分
- 设计要点
  - P2P 的架构：这区别于 Google FS 的 Single Mas 录系统的元数据



11.4 NoSQL数据库

➢Dynamo

- 设计要点
  - Performance(性能), Availability(可用性), Durability(数据持久性) 三者的折衷，可以根据应用的需求自由调整三者比例。
  - 数据定位使用一致性哈希；
  - Vector lock，允许数据的多个备份存在多个版本，提高写操作的可用性(用弱一致性来换取高的可用性)
  - 容错：Sloppy Quorum, hinted handoff, Merkle tree
  - Sloppy Quorum马虎仲裁，并非采用严格的数据一致性检查，用于实现最终一致性

11.4 NoSQL数据库

➢Dynamo

- 技术要点
  - hinted handoff(提示移交)节点故障会恢复时，可动态维护系统可用性，使系统的写入成功率大大提升。
  - 使用Merkle tree为数据建立索引，只要任意数据有变动，都将快速反馈出来。
  - 网络互联: Gossip-based membership protocol ，一种通讯协议，目标是让节点与节点之间通信，实现去中心化。
  - Storage load balancing
  - Client-driven Coordination

11.4 NoSQL数据库

➢Dynamo

- 特点
  - 高可用：设计上天然没有单点，每个实例由一组节点组成，从应用的角度看，实例提供 IO 能力。一个实例上的节点可能位于不同的数据中心内, 这样一个数据中心出问题也不会导致数据丢失。
  - 总是可写：hinted handoff确保在系统节点出现故障或节点恢复时，能灵活处理
  - 可根据应用类型优化可用性、容错性和高效性配置
  - 去中心化，人工管理工作少
  - 可扩展性较差：由于增加机器需要给机器分配DHT(分布式hash table)算法所需的编号，操作复杂度较高，且每台机器存储了整个集群的机器信息及数据文件的Merkle Tree信息，机器最大规模只能到几千台。

11.4 NoSQL数据库

➢Cassandra

- 简介
  - Apache Cassandra 是一套开源分布式NoSQL数据库系统
  - 它最初由Facebook开发，用于储存收件箱等简单格式数据，集Google BigTable的数据模型与Amazon Dynamo的完全分布式的架构于一身。
  - Facebook于2008将 Cassandra 开源，此后，由于Cassandra良好的可扩放性，被Digg、Twitter等知名Web 2.0网站所采纳，成为了一种流行的分布式结构化数据存储方案。

11.4 NoSQL数据库

➢Cassandra

- 架构要点
  - 使用了Google BigTable的数据模型　这是一种面向列的数据库，列被组织成为列族(Column Family)，在数据库中增加一列非常方便。对于搜索和一般的结构化数据存储，这个结构足够丰富和有效。
  - 系统架构与Dynamo一脉相承　是基于O(1)DHT(分布式哈希表)的完全P2P架构，与传统的基于Sharding的数据库集群相比，Cassandra可以几乎无缝地加入或删除节点，非常适于对于节点规模变化比较快的应用场景。
  - Cassandra的数据会写入多个节点　保证数据的可靠性，在一致性、可用性和网络分区耐受能力(CAP)的折衷问题上，Cassandra比较灵活，用户在读取时可以指定要求所有副本一致(高一致性)、读到一个副本即可(高可用性)或是通过选举来确认多数副本一致即可(折衷)。这样，Cassandra可以适用于有节点、网络失效，以及多数据中心的场景。

11.4 NoSQL数据库

➢Cassandra

- 特点
  - 模式灵活： 使用Cassandra，像文档存储，你不必提前解决记录中的字段。你可以在系统运行时随意的添加或移除字段。这是一个惊人的效率提升，特别是在大型部署上。
  - 真正的可扩展性： Cassandra是纯粹意义上的水平扩展。为给集群添加更多容量，可以指向另一台电脑。你不必重启任何进程，改变应用查询，或手动迁移任何数据。
  - 多数据中心识别： 你可以调整你的节点布局来避免某一个数据中心起火，一个备用的数据中心将至少有每条记录的完全复制。

11.4 NoSQL数据库

➢Redis

- 简介
  - redis是一个key-value存储系统。
  - 和Memcached类似，它支持存储的value类型相对更多，包括string(字符串)、list(链表)、set(集合)和zset(有序集合)。
  - 这些数据类型都支持push/pop、add/remove及取交集并集和差集及更丰富的操作，而且这些操作都是原子性的。
  - 在此基础上，redis 支持各种不同方式的排序。与memcached一样，为保证效率，数据都是缓存在内存中。
  - 区别是redis会周期性的把更新的数据写入磁盘或者把修改 操作写入追加的记录文件，并且在此基础上实现了master-slave(主从)同步。
- 相比memcached的优势：
  - value类型更丰富
  - 数据操作方法更多
  - 可将内存数据持久化

11.4 NoSQL数据库

➢MongoDB

- 简介
  - MongoDB是一个基于分布式文件存储的数据库。
  - 由C++语言编写,旨在为WEB应用提供可扩展的高性能数据存储解决方案。
  - 特点是高性能、易部署、易使用，存储数据方便。
  - MongoDB是一个介于关系数据库和非关系数据库之间的产品，是非关系数据库当中功能最丰富，最像关系数据库的。
  - MongoDB支持的数据结构非常松散，是类似json的bjson格式，因此可以存储比较复杂的数据类型。
  - Mongo最大的特点是支持的查询语言非常强大，其语法有点类似于面向对象的查询语言，几乎可以实现类似关系数据库单表查询的绝大部分功能，而且还支持对数据建立索引。

11.4 NoSQL数据库

➢MongoDB

- 主要特点
  - 面向集合存储：易存储对象类型的数据，包括文档内嵌对象及数组。
  - 模式自由：无需知道存储数据的任何结构定义，支持动态查询、完全索引，可轻易查询文档中内嵌的对象和数组
  - 文档型：存储在集合中的文档，被存储为键-值对的形式。键用于唯一标识一个文档，为字符串类型，而值则可以是各种复杂的文件类型。
  - 高效的数据存储：支持二进制数据及大型对象
  - 支持复制和故障恢复：提供Master-Master、Master-Slave模式的数据复制及服务器之间的数据复制
  - 自动分片：以支持云级别的伸缩性，支持水平的数据库集群，可动态添加额外的服务器

11.4 NoSQL数据库

➢功能

- 面向集合的存储：适合存储对象及JSON形式的数据。
- 动态查询：Mongo支持丰富的查询表达式。查询指令使用JSON形式的标记，可轻易查询文档中内嵌的对象及数组。
- 完整的索引支持：包括文档内嵌对象及数组。Mongo的查询优化器会分析查询表达式，并生成一个高效的查询计划。
- 查询监视：Mongo包含一个监视工具用于分析数据库操作的性能。
- 复制及自动故障转移：Mongo数据库支持服务器之间的数据复制，支持主-从模式及服务器之间的相互复制。复制的主要目标是提供冗余及自动故障转移。
- 高效的传统存储方式：支持二进制数据及大型对象(如照片或图片)
- 自动分片以支持云级别的伸缩性：自动分片功能支持水平的数据库集群，可动态添加额外的机器。

11.4 NoSQL数据库

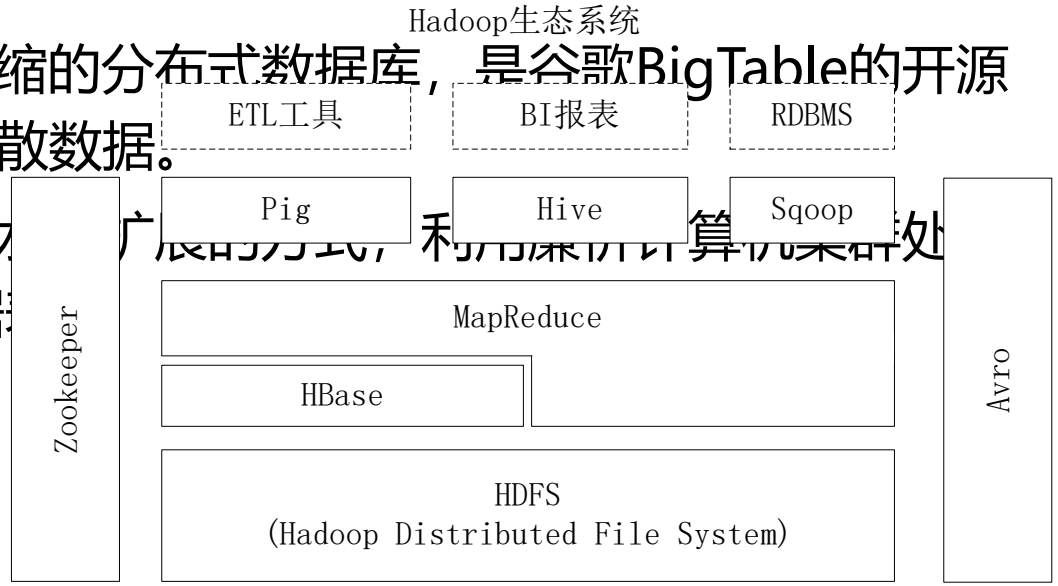
➢MongoDB

- 不适用场景
  - 要求高度事务性的系统
  - 传统的商业智能应用
  - 复杂的跨文档(表)级联查询

11.4 NoSQL数据库

➢HBase

- 简介
  - HBase是一个高可靠、高性能、面向列、可伸缩的分布式数据库，是谷歌BigTable的开源实现，主要用来存储非结构化和半结构化的松散数据。
  - HBase的目标是处理非常庞大的表，可以通过水平扩展的方式，利用廉价计算机集群处理由超过10亿行数据和数百万列元素组成的数据表



11.4 NoSQL数据库

➢HBase

- HBase和BigTable的底层技术对应关系

	BigTable	HBase
文件存储系统	GFS	HDFS
海量数据处理	MapReduce	Hadoop MapReduce
协同服务管理	Chubby	Zookeeper

11.4 NoSQL数据库

➢HBase

- HBase与传统的关系数据库的区别：
  - 数据类型：关系数据库采用关系模型，具有丰富的数据类型和存储方式，HBase则采用了更加简单的数据模型，它把数据存储为未经解释的字符串
  - 数据操作：关系数据库中包含了丰富的操作，其中会涉及复杂的多表连接。HBase操作则不存在复杂的表与表之间的关系，只有简单的插入、查询、删除、清空等，因为HBase在设计上就避免了复杂的表和表之间的关系
  - 存储模式：关系数据库是基于行模式存储的。HBase是基于列存储的，每个列族都由几个文件保存，不同列族的文件是分离的

11.4 NoSQL数据库

➢HBase

- HBase与传统的关系数据库的区别：
  - 数据索引：关系数据库通常可以针对不同列构建复杂的多个索引，以提高数据访问性能。HBase只有一个索引——行键，通过巧妙的设计，HBase中的所有访问方法，或者通过行键访问，或者通过行键扫描，从而使得整个系统不会慢下来。
  - 数据维护：在关系数据库中，更新操作会用最新的当前值去替换记录中原来的旧值，旧值被覆盖后就不会存在。而在HBase中执行更新操作时，并不会删除数据旧的版本，而是生成一个新的版本，旧有的版本仍然保留。
  - 可伸缩性：关系数据库很难实现横向扩展，纵向扩展的空间也比较有限。相反，HBase和BigTable这些分布式数据库就是为了实现灵活的水平扩展而开发的，能够轻易地通过在集群中增加或者减少硬件数量来实现性能的伸缩。

11.4 NoSQL数据库

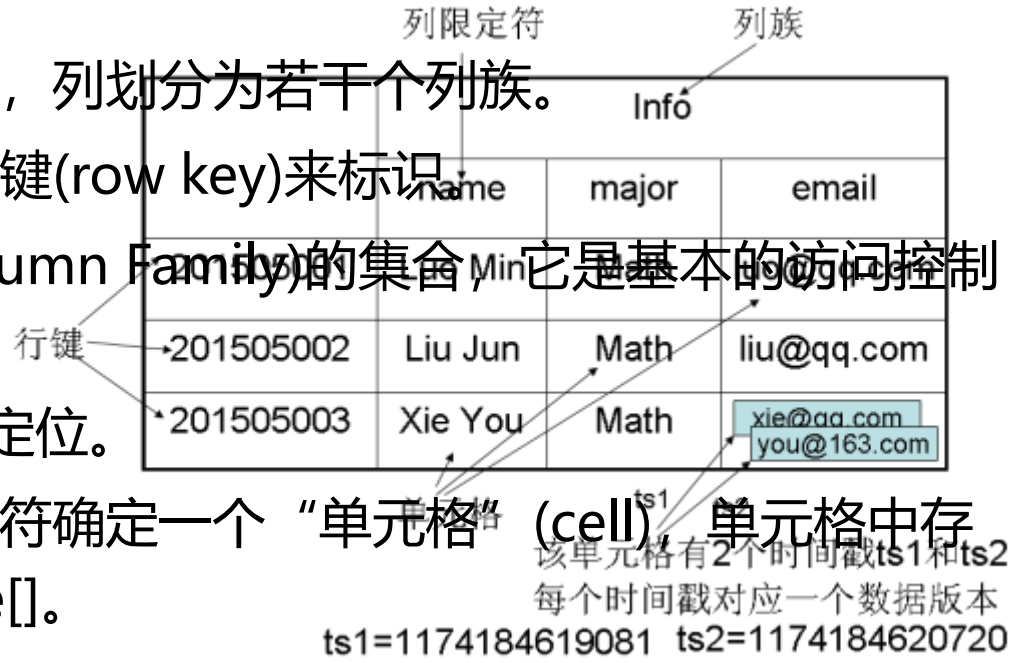
➢HBase

- 数据模型概述
  - HBase是一个稀疏、多维度、排序的映射表，这张表的索引是行键、列族、列限定符和时间戳。
  - 每个值是一个未经解释的字符串，没有数据类型。
  - 用户在表中存储数据，每一行都有一个可排序的行键和任意多的列。
  - 表在水平方向由一个或者多个列族组成，一个列族中可以包含任意多个列，同一个列族里面的数据存储在一起。
  - 列族支持动态扩展，可以很轻松地添加一个列族或列，无需预先定义列的数量以及类型，所有列均以字符串形式存储，用户需要自行进行数据类型转换。
  - HBase中执行更新操作时，并不会删除数据旧的版本，而是生成一个新的版本，旧有的版本仍然保留(这是和HDFS只允许追加不允许修改的特性相关的)。

11.4 NoSQL数据库

➢HBase

- 数据模型相关概念
  - 表：HBase采用表来组织数据，表由行和列组成，列划分为若干个列族。
  - 行：每个HBase表都由若干行组成，每个行由行键(row key)来标识。
  - 列族：一个HBase表被分组成许多“列族”(Column Family)的集合，它是基本的访问控制单元。
  - 列限定符：列族里的数据通过列限定符(或列)来定位。
  - 单元格：在HBase表中，通过行、列族和列限定符确定一个“单元格”(cell)，单元格中存储的数据没有数据类型，总被视为字节数组byte[]。
  - 时间戳：每个单元格都保存着同一份数据的多个版本，这些版本采用时间戳进行索引。

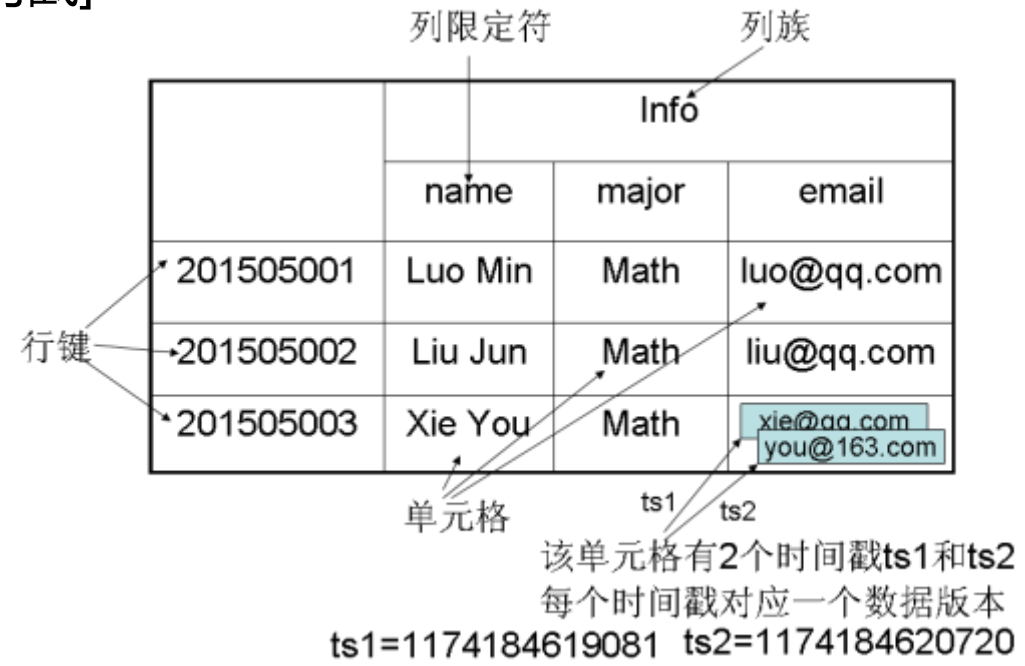


11.4 NoSQL数据库

➢HBase

- HBASE数据坐标
  - HBase中需要根据行键、列族、列限定符和时间戳来确定一个单元格，因此，可以视为一个“四维坐标”，即[行键, 列族, 列限定符, 时间戳]

键	值
[“201505003”, “Info”, “email”, 1174184619081]	“xie@qq.com”
[“201505003”, “Info”, “email”, 11741846620720]	“you@163.com”



11.4 NoSQL数据库

➢HBase

表4-4 HBase数据的概念视图

行键	时间戳	列族contents	列族anchor
"com.cnn.www"	t5		anchor:cnnsi.com=”CNN”
	t4		anchor:my.look.ca="CNN.com"
	t3	contents:html="<html>..."	
	t2	contents:html="<html>..."	
	t1	contents:html="<html>..."	

11.4 NoSQL数据库

➢HBase

表4-5 HBase数据的物理视图

列族contents

行键	时间戳	列族contents
"com.cnn.www"	t3	contents:html="<html>..."
	t2	contents:html="<html>..."
	t1	contents:html="<html>..."

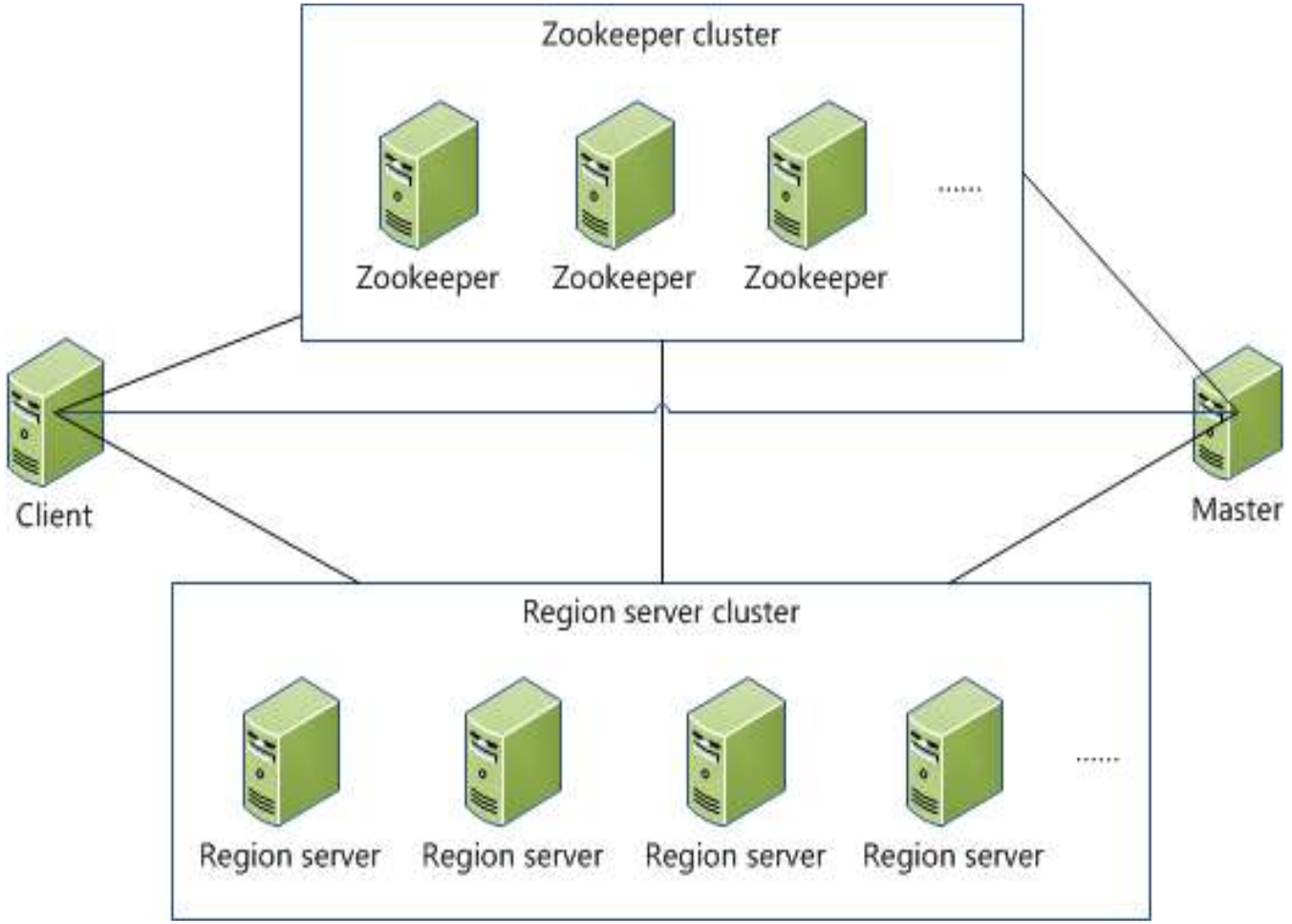
列族anchor

行键	时间戳	列族anchor
"com.cnn.www"	t5	anchor:cnnsi.com=”CNN”
	t4	anchor:my.look.ca="CNN.com"

11.4 NoSQL数据库

➢HBase

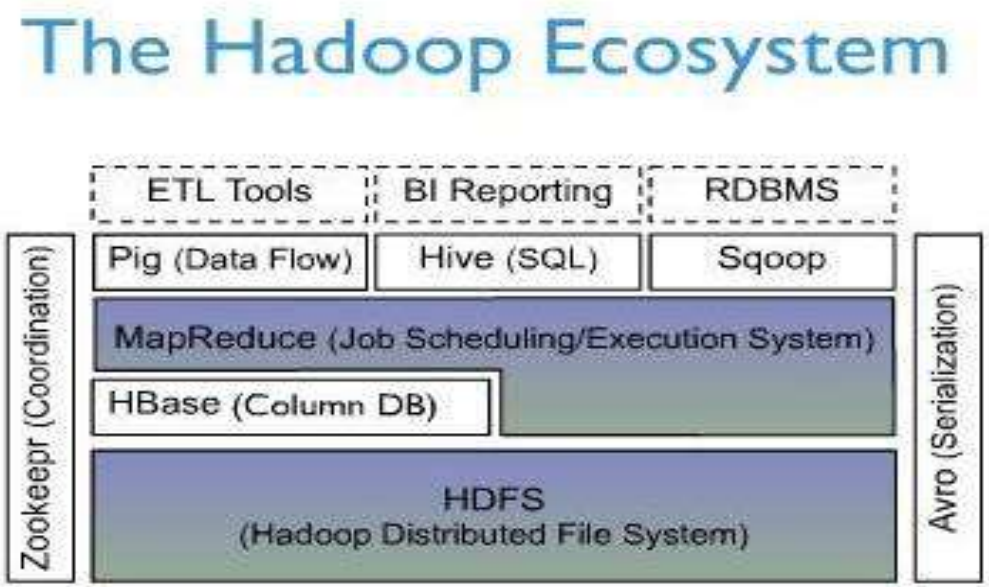
- 架构图



11.4 NoSQL数据库

➢HBase

- 架构图

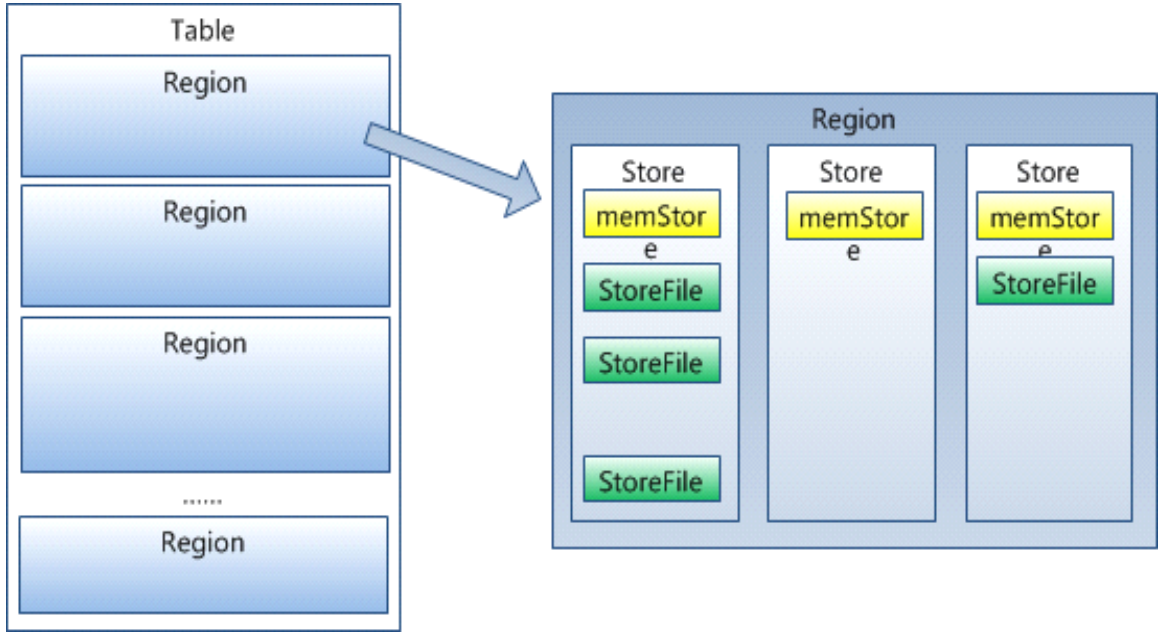


- Hadoop EcoSystem中的各层系统，其中HBase位于结构化存储层，Hadoop HDFS为HBase提供了高可靠性的底层存储支持，Hadoop MapReduce为HBase提供了高性能的计算能力，Zookeeper为HBase提供了稳定服务和failover机制。

11.4 NoSQL数据库

➢HBase

- 物理存储
  - 每个HRegion由多个Store构成，每个Store由一个memStore和0或多个StoreFile组成，每个Store保存一个Columns Family。
  - StoreFile以HFile格式存储在HDFS中。



11.4 NoSQL数据库

➢HBase

- Client 包含访问hbase 的接口，client 维护着一些cache 来加快对hbase 的访问，比如regione 的位置信息。
- Zookeeper
  - 保证任何时候，集群中只有一个master
  - 存贮所有Region 的寻址入口
  - 实时监控Region Server 的状态，将Region server 的上线和下线信息，实时通知给Master
  - 存储Hbase 的schema,包括有哪些table，每个table 有哪些column family
- Master
  - 为Region server 分配region
  - 负责region server 的负载均衡
  - 发现失效的region server 并重新分配其上的region
  - GFS 上的垃圾文件回收
  - 处理schema 更新请求

11.4 NoSQL数据库

➢HBase

- Region Server
  - 维护Master 分配给它的region，处理对这些region 的IO 请求。
  - 负责切分在运行过程中变得过大的region。
  - 可以看出，client 访问hbase 上数据的过程并不需要master 参与，寻址访问zookeeper和region server，数据读写访问regione server，master仅仅维护者table 和region 的元数据信息，负载很低。